

A Project Report On

AID-SOC: An AI-Driven for Security Operations Center

Submitted in partial fulfillment of the requirement for the
award of the degree

**MASTER OF SCIENCE IN
CYBER SECURITY AND CYBER LAW
M.Sc. (CS & CL)**

Academic Year 2025 - 26

Surya Suresh (92400565053)

Internal Guide
Dr. Pankaj Mudholkar



Marwadi
University
Marwadi Chandarana Group



Rajkot-Morbi Road, At & PO : Gauridad, Rajkot 360 003. Gujarat. India.



Marwadi
University
Marwadi Chandarana Group



Faculty of Computer Applications (FCA)

Certificate

This is to certify that the project work entitled

submitted in partial fulfillment of the requirement for

the award of the degree of

MASTER OF SCIENCE IN

CYBERSECURITY AND CYBERLAW

M.Sc. (CS & CL)

of the

Marwadi University

is a result of the bonafide work carried out by

Surya Suresh (92400565053)

during the academic year 2025-26

Faculty Guide

HOD

Dean

CERTIFICATE

DECLARATION

I hereby declare that this project work entitled **AID-SOC: An AI-Driven for Security Operations Center** is a record done by me.

I also declare that the matter embodied in this project is genuine work done by me and has not been submitted whether to this University or to any other University / Institute for the fulfillment of any course of study.

Place:

Date:

Surya Suresh (92400565053) **Signature:** _____

ACKNOWLEDGEMENT

It is indeed a great pleasure to express my thanks and gratitude to all those who helped me. No serious and lasting achievement one can ever achieve without the help of friendly guidance and co-operation of so many people involved in the work.

I am very thankful to my faculty guide **Dr. Pankaj Mudholkar**, the person who makes me follow the right steps during project work. I express my deep sense of gratitude to his guidance, suggestions and expertise at every stage. Apart from that his/her valuable and expertise suggestion during documentation of my report indeed helped me a lot.

Thanks to my friends who have been a source of inspiration and motivation that helped me during my project work. I am also thankful to my HoD **Dr. Sunil Bajaja** and Dean **Prof. (Dr.) Rijwan Khan** for their support as and when required. I would like to extend my gratitude towards all who directly or indirectly supported and helped me to accomplish my project work.

Surya Suresh (92400565053)

Signature : _____

CONTENTS

Chapters	Particulars	Page No.
1	SYNOPSIS	1-5
2	PREAMBLE	6-22
2.1	General Introduction	
2.2	Statement of Problem	
2.3	Objective and Scope of the Study	
2.4	Module Description with functionality	
2.5	Methodology	
2.6	Feasibility Study	
3	REVIEW OF LITERATURE	23-28
3.1	Study of Existing Similar Types of Systems	
4	TECHNICAL DESCRIPTION	28-32
4.1	Hardware Requirement	
4.2	Software Requirement	
5	SYSTEM DESIGN AND DEVELOPMENT	34-45
5.1	Architectural Design / System Flow	
5.2	Data Flow Diagram (DFD)	
5.3	Structural Modeling	
5.3.1	• Class Diagram	
5.4	Behavioral Modeling	
5.4.1	• Use Case Diagram	
5.4.2	• Sequence Diagram	
5.4.3	• Activity Diagram	
5.5	Database Design	
5.5.1	• Table Structure	
5.5.2	• Entity Relationship Diagram	
5.6	Menu Design	
5.7	Screen Design	
6	SYSTEM TESTING	46-57
6.1	Testing and Implementation	
6.2	Testing Methodology	
6.3	Test Case Design (at least 10)	
6.4	System Implementation Strategy	
7	CONCLUSION	58-60
8	LEARNING DURING PROJECT WORK	61-64
9	FUTURE ENHANCEMENTS	65
10	BIBLIOGRAPHY	66-69
10.1	Online References	
10.2	Offline References	

FIGURE INDEX

Sr. No.	Figure No.	Particulars	Page No.
1	5.1	System Flow Diagram	36
2	5.2	Data Flow Diagram (L0)	37
3	5.3	Data Flow Diagram (L1)	37
4	5.4	Class Diagram	38
5	5.5	Use Case Diagram	39
6	5.6	Sequence Diagram	40
7	5.7	Activity Diagram	41
8	5.8	ER Diagram	43
9	5.9	Menu Design	43
10	5.10	Dashboard (Overview)	44
11	5.11	Alert Menu	44
12	5.12	AI Engine (1)	45
13	5.13	AI Engine (2)	45
14	5.14	AI Engine (3)	46
15	5.15	Threat Trends in 24 hrs	46
16	5.16	Alert Distribution	47
17	5.17	AI Performance	47
18	5.18	Top Ranked Threats	48

TABLE INDEX

Sr. No.	Table No.	Particulars	Page No.
1	3.1	Comparative Study of Related Research	30
2	4.1	Processor	32
3	4.2	Memory	32
4	4.3	Storage Requirements	33
5	4.4	Network Requirements	33
6	4.5	Scalability Consideration	33
7	4.6	Operating System Requirements	34
8	4.7	Python Environments Requirements	34
9	4.8	Core Python Dependencies	34
10	4.9	Web Browser Requirements	35
11	4.10	Integration Capabilities	35
12	5.1	Alerts Table	42
13	5.2	Session State Structure	42
14	5.3	Isolation Forest	42
15	5.4	DBSCAN	42
16	6.1	Major Test Cases	55

CHAPTER – 1

SYNOPSIS

Watching who logs in helps some groups fight digital break-ins - this tool uses smart software to watch patterns. Not stuck with fixed checklists, it studies normal entry habits, then flags odd behaviours by itself. Big teams lean on tight login barriers since they protect key files and work systems. Still, intruders keep slipping through, testing password after password, reusing leaked ones, searching for open doors. Each try adds lines to logs, yet pulling out threats from mountains of data needs sharper senses than people alone offer. Midstream through activity streams, unusual patterns get marked right away instead of waiting weeks. The shift? Warnings show up before trouble spreads - timing changes everything.

Most Security Operations Centers rely on preset rules to catch threats, often built around known behaviours or clear thresholds. When fresh attack styles appear, though, those systems fail to notice subtle changes creeping in. Because old detection methods still run, alerts flood in - many false, most unhelpful. Rather than depending only on rigid checks, a different path emerges through techniques such as Isolation Forest and DBSCAN, drawn from unsupervised machine learning. Patterns in login timing get examined closely, focusing on flow instead of single events. Hidden within ordinary activity, strange sequences reveal themselves when viewed this way.

Fresh event records from Windows security stay central here, focusing on login attempts flagged as 4625 if denied, 4624 when cleared, along with 4740 at logout moments. Real logs alone feed the process - no test runs slip through. Instead of recycled files or artificial triggers, activity flows directly from live system signals. Since analysis waits for new entries to appear, results stick close to how things actually run. That raw stream keeps everything tied to what's really happening. When copies are present, outcomes may skew. To prevent that, monitoring live interactions blocks reprocessing the same entries. Because status checks stay steady, studying incoming flows becomes quicker. What matters is how smoothly data moves without hiccups.

Starting different sections build up slowly, AID-SOC runs on pieces that link together. Checking logs every so often replaces endless watching, grabbing just what shows someone trying to get in. Login frequency becomes the core focus - too many failed tries show up clearly, sudden wins after mistakes raise flags, strange hours feel out of place when patterns

shift. Normal rhythm acts like a baseline; deviations stand out without needing preset conditions. Unusual actions are not identified through any strict checklists. Rather, isolation forest techniques identify what does not match the normal behaviour. DBSCAN and similar methods find pattern of behaviour. These group of actions that happens can come from the same source. This suggests shared risks. Multiple methods are worked together. They will improve accuracy while keeping false signals at the same level.

Risk levels –Critical, High, or Medium— are identified by an automated generated score. This scoring evaluates mainly 3 factors which are the severity of the event, its changes from the normal behavioural patterns, and the surrounding factors. Because the higher score indicated greater risks and that should be prioritized for the review purpose. The system processing these warnings are removed by duplicates, labelling their present status, and storing them in order. Performance metrics which includes alert volume and current threat trends are channeled into monitoring tools. Down the road? Other threats may fall into view just as easily.

Watching login attempts helps some groups fight digital break-ins - this screen handles that job with smart software. Not stuck to fixed patterns, it studies normal entry habits, then flags odd behaviour without being told. Big teams lean hard on protected doors since they block paths to key data and systems people use every day. Still, intruders keep slipping through, testing thousands of passcodes, reusing leaked ones, or hunting live profiles. As attacks roll in, records grow fast; catching threats hidden across heaps of lines needs something faster than a person scanning alone. Right when something unusual happens, the system notices - no waiting till next week. Most methods catch problems late; this one speaks up early. While things are still moving, it reacts.

By implementing algorithms like Isolation Forest and DBSCAN, the system can now analyze the login activity over time without any list of what an attack looks like. How actually it works is instead of searching for isolated events, the system examines the “flow” of user activity. It also identifies the behavioural change that is opposite to the everyday routines. Threats are not only detected by one suspicious action but also by analysing the user behavioural pattern.

The system follows live Windows Security event logs to monitor authentication activity in real time and it mainly focuses on the event IDs to login lifecycle are successful login (4624), failed login attempt (4625), account lockout (4740). To maintain the data integrity and accuracy the system strictly processes live telemetry. This approach ensures the results shows the actual behaviour on the running device. It won't make any kind of assumptions

and gives the results from the present activity. Also, the system removes the duplicate entries to prevent the redundancy and keep data clean. By maintaining the flow of the incoming data, the system follows the processing speed without any sacrifices. This real-time monitoring ensures that even high volume of login attempts can be handled efficiently without any system lag.

- **The Systems Building and Organization.**

Python allows the processing of background events to occur concurrently with interactions on the dashboard to ensure that the system remains responsive. This location is concerned with effective memory management as the old data are cleared automatically to allow new data as far as the trend analysis is concerned. The data is stored in readable by human beings in the form of JavaScript Object Notation files. It employs locking options and temporary temporary swap files to circumvent corruption in the presence of the access or a crash in the system. The system educates machine learning models to detect the behavior of a bot by converting the logs of the logs into numeric variables, e.g., time of day, number of attempts, and failure rates, to make the system classify common patterns. Probing revealed that the contamination value of 0.1 was a combination offering an ideal balance between the Isolation Forest algorithm; the lower contamination value exceeded the accurate red flags, whereas higher values produced an excessively high level of noise. This environment was discovered by trial and error using real world logs, which reflected that real world data is usually more accurate than mere formulas.

- **Applying Systems to Work in the Real World.**

It is operational at once and has facilities of isolated environments to separate project tools and system programs. The application will promote the uniformity of behaviour of various workstations. High permissions allow it to get secured Windows logs. The tool has step-by-step instructions on how to enable these logging and enhance tracking.

The performance is based on hardware available. As an illustration, with the size of 4GB of free ram in a typical desktop, it is possible to serve up 200 events/min and the response time is less than three seconds. With the increasing volume of logins, the upgrade of memory and processing force ensures that the system remains with the real usage patterns. The dashboard receives new data every three seconds, so as to strike a balance between the speed and system stability. This gives an immediate response without straining the server. Whenever a connection is lost, the system is upgraded to reconnect automatically with the user being prompted on the status. The interface has a smooth transition of updating graphs

to assist analysts focus on trends of data emerging without any sudden changes in the interface.

Also, the alert workflow reduces efforts in redundancy as it lets the analysts use specific alerts and record findings in permanent closing reports. This shared ability that allows bulk-processing associated alerts accelerates the response time and creates a consistent audit trail, to use in compliance and pattern verification.

It is important to note that the level of education and transfer of knowledge can be enhanced through the implementation of this methodology in the educational process:

In addition to the activity of the monitoring system, this work demonstrates the operation of AI in the actual security conditions. Reflections on machine learning can be shown in ways that students can see that are applicable in real-life situations and not just theory. The modular design of the system makes them see the simple ideas about the split of the task, interface design, and components. Furthermore, the code contains comments that elaborate as to why certain technical options are made. This will enable the users to understand the reasons behind this or that method rather than seeing the end product.

The system is used in a classroom, as a practical laboratory where the users can test the AI behaviour. Students can understand sensitivity settings by switching the alert limits, which makes them generate more false positives or cause the missed critical threats. This, in contrast with artificial bits, allows students to prepare in terms of realistic code by using their own rules or display tools, which makes them practical in an environment that is neither simulated nor predictable. The fact that you can easily deal with such live data feeds reveals how fluke real world information can be. Like memory leaks and timing glitches are issues that should be disputed by students. This continuous process alters the conception of what a system can be considered to be in a stable condition, educating the user how to keep the performance stable and transferring the input.

- **What Comes Next Can Grow**

The modular nature of the system provides a strong foundation upon which it is possible to substantially increase security surveillance without disrupting the current operations. With the support of the new types of events, the platform can trace illegal access to the file, malicious processes, or concealed network channels by attackers. Regardless of the place of origin of data, all checks are subject to a workflow. The system monitors the incident, then uses intelligent software and displays the findings on an easy-to-understand dashboard. This arrangement enables the extension of monitoring to Linux and MacOS by specific intake layers that are fed by tailored intake layers which are integrated into a single

central detection engine. This is a cross-platform strategy that provides an overall picture of the level of risk within the entire network, providing a consistent security position, with or without regard to the operating system being used.

The system is built to share alert data and invoke automated responses using the REST APIs and webhooks to operate in a larger security environment. It also has the ability to relay logs over syslog to a central collection location, making the platform appear like a networked security layer rather than a single tool. The deep learning and attention mechanisms can significantly help improve the performance in the future because they are efficient to detect subtle and long-term threats that emerge gradually. The models can also adapt to new environment without necessarily going through the training process again upon the transfer learning. Finally, the system provides an intelligent yet unobtrusive supervision tool that can be filtered to the essential security indicators. Rather than detailed reports, responses are presented on white screens that are easily tracked by anyone. Since it is used on what teams encounter in their day-to-day activities, there is no need to force it. Knowledge is silent - every single piece of knowledge accumulates. As systems can identify patterns quicker than humanities, confidence will develop off the record. Development in this case does not scream but just manifests itself in a regular manner. Each time another step is taken with the help of an alert being reviewed, the reaction of old ways of reacting too late passes past.

CHAPTER – 2

PREAMBLE

2.1 General Introduction

Digital tools will assist companies in sharing the information, data storage, financial management, and work organization. The more connecting, the more cyber threats becoming more difficult and cleverer each year. Malicious people constantly also evolve their approaches, and they do not rely on old-fashioned security practices that were once successful. As a reaction, special units called SOC's control the work and detect problems, evaluate risks and react in case of security breaches. Nonetheless, a lot of these procedures are based on pre-determined regulations and reviewing logs manually; this method cannot keep pace with the rapid pace of contemporary attacks.

This type of approach can dishonour even when they received familiar threats and are powerless against emerging or unfamiliar threats. In addition to this, alerts appear and most of them are false positives. Since the false alerts were given, genuine problems were left unnoticed. The solution of this issue is smart monitoring: it reports abnormal behaviour, interprets the habitual behaviour and then silently helps investigators without any notifications.

A new form of security involves the monitoring of threats on a real-time basis using software. It assists in the analysis of the unfolding events. The first indications of trouble are excessive incorrect guesses, account attempts that do not match and lockouts that appear at the same time. Such instances produce wake-up calls.

Abnormal behaviour is strange, through clever identification, based on machine learning. It does not make use of predetermined checklists it evolves new forms as situations evolve. Live updates will be on the screen and will not have any delays and any other unwarranted filters. As an alert already gives a clear context users can quickly determine the urgency, severity. Early security monitoring has changed over time. Many authentication challenges faced. Real time processing has been improved over time.

Clear dashboards enhance decision-making. Complex interfaces make unfair with the response efficiency. Simple layouts support faster investigations. Visual clarity boosts threat recognition. Alerts provide user identity details.

2.2 Statement of the Problem

A steady stream of alerts creates issues. Each time specific conditions trigger—like multiple failed sign-ins—a notification goes off. While good at spotting clear attack patterns, they overlook background details. Shifts in normal behavior or unusual hours often slip through. So many alarms sound even when nothing is wrong. Overwhelmed by false alarms, an analyst gradually stops responding. As warnings accumulate, focus diminishes—key alerts get missed. A stretched mind overlooks what it should catch.

Smart hackers today use subtle tactics instead of loud break-ins. Hours pass between their attempts, making bursts seem harmless at first glance. Different accounts are tested one after another, quietly changing identity each time. Access that appears legitimate is actually twisted for hidden goals. Simple alarms miss these moves because nothing crosses a set limit. Patterns are now more important than isolated red flags. When danger hides within everyday behavior, old rules fail without context. Observing how people typically act helps identify what stands out.

Finding skilled cyber defenders is not easy, no matter where you look. Staff shortages mean businesses often miss what happens in their systems overnight. That's why smart software now steps in, identifying strange actions without human help. If left unnoticed, dangers can linger—costs rise, reputations suffer. Speed is crucial when spotting trouble.

Building a real-time monitoring system means accurately identifying unusual login behaviors and clearly displaying results on a simple SOC screen.

- **Challenges in modern enterprises**

A major changes and improvements have been continuously happening over year in the modern enterprises. As improvements several challenges are also facing.

If people need to check things by hand now and then, that approach falls apart quickly.

- **The False Positive Problem**

Most old-school alert systems struggle to catch real threats without overwhelming teams with noise. When alerts are too broad, they identify risks but also raise false alarms frequently. Tighter filters reduce false alerts, yet they can let sneaky breaches go unnoticed. Finding the right balance rarely works for every situation. What works for one company might fail for another.

Investigating each alert requires someone to look into what happened, check supporting

records, and determine if a real threat exists. Even a small error rate—like one in ten—means most time gets spent chasing nothing. Pursuing these false alarms delays the detection of real attacks.

- **Resource Limits Meet Real World Operations**

Many teams face the same risks as larger companies but can't invest in the necessary systems. Without the right tools, smaller teams remain vulnerable. Security becomes inconsistent when cost limits access.

- **The Need for Accessible Solutions**

Something tough affects schools every day. Working with limited funds, universities must balance protecting personal student information with managing ongoing research records. Keeping an eye out for threats is important, but top-tier systems often come with high costs. Here's an interesting point: when safety technology also serves as a classroom tool, it can do both jobs without requiring extra resources.

Free software demonstrates that high-quality tools don't need large budgets. Starting with code anyone can use, experts create systems and share them widely. When one group improves a tool, others benefit without needing to start from scratch. Progress happens faster because fixes and ideas flow naturally between teams. What begins as a single project grows stronger as many people shape it in their own ways. Strength develops when people share their work openly instead of keeping it to themselves.

However, open-source security tools must meet certain quality standards before people will actually use them.

- From this analysis I mentioned that my problem statement is” Educational institutions are basically facing struggle to provide practical SOC training because of expensive SIEM tools are costly. As a result, students are only been taught theory. So to get exposure to the practical knowledge about how SIEM works I proposed the project named AID-SOC Dashboard”

2.3 Objective and Scope of the Study

- Real-time Authentication Monitoring: Monitor Windows login activity instantly and detect unusual login behavior as it happens. Use adaptive learning instead of fixed rules.
- Machine Learning Implementation: Isolation Forest and DBSCAN to analyze live Windows login events (4624, 4625, 4740).
- Build we based dashboard.
- Generate alerts within 3 seconds.
- Analyze user performance for reduce false positives.
- Detect authentication security-based attacks only.
- This is developed for educational purpose.
- AI-detection with rule-based detection included.
- Generates only Windows Login Events.
- Needs admin rights
- Dashboard runs smoothly.
- According to SDG-4, SDG-9, SDG-4.4 and SDG-9.5 it included hands on cybersecurity tools and skills for the students. It also includes the industry, innovation and infrastructure. It tells the job opportunities for the students and cyber professionals and the innovation and technologies used for the modern technologies.

2.4 Module Description with Functionality

- **Log Ingestion Module - Detailed Architecture**

The ingestion module connects to Windows through the pywin32 library, which allows Python to access native Windows APIs. It uses Event Log API functions like EvtQuery and EvtNext to list security events. Working at this low level reduces latency and avoids unwanted filtering that can occur with higher-level tools.

State management is essential. Without it, the system would re-scan history every cycle, wasting CPU and I/O resources and risking duplicate alerts. The module saves the timestamp of the most recently processed event in a small JSON file. On each pass, it only queries events that are newer than that timestamp; this is true incremental processing. Why re-scan items you've already handled?

Error handling is practical. The Windows Event Log service can be temporarily unavailable, files can become corrupted, or permissions can block access. Instead of crashing, the module logs the issue and continues running, retrying on later scans. This resilience ensures that brief issues do not stop monitoring completely.

Filtering occurs at multiple stages to reduce early workload. The initial query targets the security event log and specific authentication-related event IDs to prevent irrelevant noise from clogging the pipeline. Additional checks on event metadata eliminate system-generated items that are not user authentication attempts. Multi-stage filtering means noise is removed as soon as possible.

- **Detection Engine Module - Analysis Workflows**

Failure-pattern analysis looks at sequences of failed and successful attempts. Multiple failures followed by a success often suggest credential guessing. This workflow correlates events by account and timestamp, flagging sequences that match known attack profiles. Context is important: an occasional mistyped password followed by a success is normal, but many failures followed by a success is a red flag.

Temporal analysis compares logins against a user's typical behavior. Each user's history helps create expected login windows. Authentications that occur well outside these windows raise flags for anomalies. This requires keeping per-user temporal profiles and updating them gradually as new data comes in.

A correlation engine connects events that seem harmless on their own but form a pattern when combined. For instance, failed logins spread across many accounts in a short time hint at password spraying. The engine maintains sliding windows of recent events and inspects them for patterns across users or systems.

Machine learning comes into play through an AI scorer, but the detection engine prepares the features. It extracts metrics, such as authentication frequency, failure counts, and time-of-day and day-of-week indicators. It then provides these engineered features to ML models that learn normal behavior and identify deviations.

- **AI Scorer Module - Machine Learning Implementation**

An IsolationForest from scikit-learn is used for detecting anomalies with tuned parameters. The contamination parameter is set to 0.1 to reflect the expected proportion of anomalies. The ensemble size, or `n_estimators`, balances accuracy and processing power. Testing

showed that 100 estimators is a reasonable compromise.

Feature scaling occurs before inference. This ensures that no single metric overshadows the others. Authentication frequency and failure counts can reach into the hundreds, while time-of-day ranges from 0 to 23. Standard scaling, which uses zero mean and unit variance, prevents high-magnitude features from dominating distance-based calculations.

DBSCAN clustering groups related authentication events that may suggest coordinated activity. The epsilon parameter controls the maximum distance between members of a cluster. If it is too small, related events become fragmented, and if it is too large, unrelated events merge. Minimum samples is set to three, which means isolated events are treated as potential anomalies rather than clusters.

Confidence scoring combines rule-based detections and machine learning outputs into a single set of probabilities. Rule matches receive high scores by default because they correspond to known attack signatures. Machine learning anomaly scores carry more weight when deviations are significant. The final confidence is a weighted average normalized on a scale of 0 to 100.

- **Alert Manager Module - Lifecycle Management**

The alert manager handles everything from creation to resolution. When the detection engine finds suspicious activity, it sends the details to the manager. The manager assigns a unique ID, timestamp, severity, and initial status before saving the alert.

Deduplication prevents analysts from being overwhelmed by noise. If a new alert is similar to a recent one, such as the same user, attack type, and timing, the manager suppresses it instead of creating a new entry. Suppression windows usually last several minutes, which helps reduce noise while allowing new attack phases to be noticed.

Status tracking keeps alerts moving through investigation. New alerts begin as "Active." Analysts can acknowledge them to take responsibility and change their status to "Acknowledged." After remediation, they can mark them as "Resolved." This straightforward state system supports teamwork and accountability.

Alerts are stored in JSON format for clarity and ease of use. Each alert is a JSON object that includes a timestamp, severity, affected user, attack type, AI confidence score, and status. When the manager starts up, it loads the list of alerts into memory for quick access and regularly saves changes to disk. For this project's size, using memory is fast and practical; larger deployments would probably shift to a proper database.

A query interface allows the dashboard to request alerts based on severity, status, time

range, or user. Since queries run against the in-memory list, responses are quick, and the user interface remains efficient.

- **SOC Metrics Module - Performance Analytics**

The SOC metrics module summarizes alert data into statistics that show trends in security posture. Analysts can focus on high-level metrics instead of individual alerts. This helps with daily operations and longer-term decisions.

Alert volume metrics present totals over hourly, daily, and weekly periods. Sharp spikes may indicate attack campaigns or misconfigurations that lead to false positives. Gradual rises could suggest increasing attack activity or broader monitoring coverage. These trends help determine whether security posture is improving or declining.

Severity distribution categorizes alerts by level. Environments with many critical alerts face urgent problems. Predominantly low-severity alerts may indicate noisy detection or minor hygiene issues. This distribution provides a quick understanding of current threat intensity. Detection performance metrics evaluate how well the system identifies threats. These include detection rates for known patterns, false positive rates, and the average time from attack start to alert issuance. Tracking these metrics over time shows if detection improves as machine learning models receive more training data.

- **Web Dashboard Module - User Interface Implementation**

The web dashboard presents the user interface using Flask for backend routing and Jinja2 for templates. It follows an MVC-like separation, ensuring that data access, business logic, and presentation remain distinct. This setup prevents frontend changes from requiring backend rewrites and vice versa.

The main view provides a compact security overview through several panels: system health and monitoring status, alert summaries by severity, recent alerts with key fields, and charts visualizing trends and distributions. Together, these panels offer a quick snapshot and clear paths for deeper investigation.

Real-time updates use simple JavaScript polling. Every three seconds, the frontend queries backend APIs for current alert counts, recent alerts, and chart data. When responses arrive, the page updates without a full reload. It's not the flashiest method, but it's dependable and easy to debug.

2.5 Methodology

The AI-SOC didn't launch all at once. It was built slowly, piece by piece. Each module was designed, coded, and validated individually before being added to the larger system. Think of it like constructing a machine from carefully tested parts; you wouldn't place an engine in a chassis without testing it first. During development, we fed real Windows security alerts into the system, ensuring our tests reflected everyday noise and oddities. Live event streams guided adjustments at every step, keeping detection focused on real behaviour instead of neat lab examples.

Our detection strategy begins with actual human behavior. Instead of relying on pre-labeled datasets, the system observes changes in routine and flags any deviations. It doesn't need hand-tagged examples to spot something unusual; over time, it tracks login frequency, links related events, and compares the recognized patterns to what is typical. We kept the number of moving parts small and simple where we could, which reduces operational overhead without sacrificing accuracy.

Modules were developed independently, but their responsibilities remained clearly defined. Well-defined interfaces connect components and make troubleshooting easy. One module continually monitors events, while another activates only when anomalies surface. Automated checks run in the background to ensure new alerts keep coming. Updates arrive through standard web connections, so the dashboard remains up-to-date as the system improves.

Practical experiments using real login activity generated the most valuable signals. We observed how the software managed repeated failures, account lockouts, or late-night authentications from unfamiliar IP addresses. Validation focused on whether events were categorized correctly and if those classifications made sense. Above all, consistency under repeated tests mattered most; steady, reliable detection is more valuable than flashy one-time results.

We took an iterative approach instead of trying to design the entire system upfront. Early iterations were intentionally simple: we collected Windows events, stored them, and displayed them in a basic web interface. There was no AI at that point; we simply needed to prove that the plumbing could reliably carry security events end-to-end.

Each later iteration added new features while keeping existing ones stable. The second build introduced straightforward rule-based checks for obvious patterns, like multiple failed logins. The third added Isolation Forest for unsupervised anomaly detection, and the fourth

included DBSCAN clustering. By gradually increasing complexity, we were able to test and validate each new feature before introducing the next one.

This approach revealed problems that would have been difficult to diagnose later. Early testing showed that the event collector reprocessed historical logs during each scan and created duplicate alerts. This finding led us to add session-state tracking before implementing advanced detection. If we had piled on complexity at the start, we would have buried that duplication bug in more code, making it much harder to isolate.

The process also shaped our learning curve. Initial ML runs used default algorithm settings and performed poorly. These tests pushed us to fine-tune parameters and engineer features. Each iteration incorporated those lessons, improving detection accuracy. A waterfall approach would likely have missed many incremental insights that only experimentation reveals.

We opted for unsupervised learning for both practical and philosophical reasons. Supervised models require labeled examples of normal and malicious behavior, which are costly to produce. This involves either manually labeling thousands of events or creating fake attack scenarios. Both tasks take time and often lack real operational context.

Unsupervised methods avoid this requirement by identifying structure in unlabeled data. For instance, Isolation Forest models typical authentication behavior and highlights events that deviate from it. This aligns with our zero-simulation philosophy: we want the system to learn from real operations rather than constructed scenarios.

Unsupervised approaches do pose their own challenges. Without labels, standard validation metrics don't apply directly. We tackled this with controlled tests, where we deliberately generated certain attack patterns and measured detection rates. While this isn't a replacement for large labeled datasets, it provides reasonable confidence that the algorithms identify the kinds of threats we care about.

Establishing a baseline is essential. Models need a warm-up period to observe enough normal behavior before they can reliably flag anomalies. We keep the system in a no-alert training mode until it has collected roughly 100 events; only then does active detection begin. This prevents a wave of false positives while the model is still getting acquainted with its environment.

- **Feature engineering**

Turning raw authentication logs into meaningful features required careful, often manual, thought about what really separates attacks from normal behavior.

Temporal features capture when activity occurs. Simple fields like hour-of-day (0-23) and day-of-week (0-6) help the system identify oddities. For example, someone who usually logs in at 10 a.m. suddenly authenticating at 3 a.m. is suspicious. Encoding these as numeric values allows distance-based algorithms to use them effectively.

Frequency features measure rates, including events per hour, per day, and rolling averages across different time periods. Sudden spikes often suggest automated attacks, while gradual shifts might indicate legitimate role changes or new work schedules.

Failure-rate features compare failed attempts to total attempts. Everyone mistypes a password occasionally, and a few failures are normal. However, when failure rates exceed certain thresholds, that pattern indicates credential guessing. We record both absolute failure counts and proportional rates to capture these details.

Sequential features examine short histories, such as how many failures occur before a success, gaps between attempts, and alternating failure-success sequences. These patterns provide context that single events rarely offer, so the system maintains a compact event history for each user.

Feature scaling is also important. Without it, features with large numeric ranges can overshadow those with smaller ranges. Standard scaling, or adjusting to zero mean and unit variance, helps level the playing field and noticeably improves algorithm performance.

- **Parameter tuning**

Algorithms don't tune themselves. Parameters like contamination in Isolation Forest or epsilon in DBSCAN greatly affect outcomes, so we used a careful tuning process instead of relying on guesswork.

For Isolation Forest, contamination, which is the expected proportion of anomalies, needed thorough testing. If it's too low, you miss attacks. If it's too high, you get swamped with false positives. We tested contamination values from 0.05 to 0.20 in 0.025 steps. We measured detection and false-positive rates against a test set that included both normal patterns and simulated attacks. Our goal was balance: catch threats without raising alarms on routine activity.

DBSCAN tuning focused on epsilon, the maximum distance between cluster members. A small epsilon splits related events into many clusters, while high values group unrelated events together. We experimented with epsilon values from 0.3 to 0.7, using silhouette scores and manual checks to assess cluster coherence.

The `min_samples` parameter defines the minimum number of events needed to form a

cluster. This also needed adjustment. If it's too low, noise appears as a cluster. If it's too high, small but important clusters disappear. We found that three minimum samples provided a practical balance for authentication events.

Tuning led to some unexpected results. For instance, higher contamination values did not always improve detection. Moderate values around 0.10 often produced better true-positive rates. This illustrates why systematic testing is more reliable than intuition.

- **Testing and validation**

Validation combined simulated attacks with observation of normal operations. We needed to be sure the system would detect attacks without confusing routine behavior for malicious activity.

The attack simulations included brute-force sequences, where many failures are followed by success; password-spray patterns, which consist of failed attempts spread across different accounts; and account-lock scenarios that trigger Windows lockout behavior. Each simulation had an expected response that we compared to the actual output.

At the same time, we monitored long runs under normal usage for false positives. A system that frequently signals a problem is nearly useless. Thus, measuring and explaining false positives was a key part of our validation process.

Performance testing looked at resource use and latency under various loads. We created events ranging from light (10 events per minute) to heavy (500 events per minute) and tracked memory, CPU, and alert-generation latency to make sure the pipeline could handle the expected operational range.

Sustained operation tests ran the system for days or weeks to uncover issues that shorter tests might miss, such as memory leaks, slow degradation, and resource exhaustion. These long-duration trials confirmed that the platform remains stable in actual deployments.

- **Integration testing**

Unit tests checked modules on their own, but integration tests confirmed the complete process. This included collecting events, analyzing them, scoring them with AI, turning them into alerts, and showing them on the dashboard.

End-to-end traces created authentication events and tracked them through each component. This process involved checking accuracy at each step and ensuring the pipeline met latency standards. We also conducted error-propagation tests by injecting faults, such as event log

access issues, file write errors, and network disruptions. This ensured the system logs failures and reduces performance gracefully without crashing.

Concurrency tests were important as well. Multiple processes can read or write the same files at the same time. We confirmed that file locking and other precautions prevent corruption when processing events and handling user requests simultaneously.

- **Documentation**

Good documentation accompanied development and targeted both operators and learners. Code-level documentation used Python docstrings to explain modules, classes, and functions. It described what they do, their parameters, and return values. Inline comments clarified non-obvious logic that might confuse future maintainers.

User documentation offered step-by-step instructions for installation, configuration, and operation. We assumed readers had basic Python knowledge but not deep expertise. Thus, the documentation included clear setup steps, dependency instructions, and initial-run checks.

Architecture documentation outlined design reasons and component interactions. Diagrams illustrated data flow and module relationships, helping newcomers quickly understand the system's structure.

Testing documentation listed procedures, expected outcomes, and actual results. This allows others to reproduce tests and check behavior in their environments. It also serves as a record that supports claims about system capabilities.

The iterative development cycle, systematic testing, and detailed documentation created an AI-SOC that meets its goals while remaining easy to deploy and modify in the future. By prioritizing practical validation over theoretical assumptions, the project provides real confidence that the system works in actual operational settings.

2.6 Feasibility Study

Available tools, Python, Flask, and the Windows Event Log API, make this project easy to build. These are familiar, well-documented technologies used throughout the industry, so you are not creating an unusual stack just to make the system work. In everyday use, they are stable; they handle authentication logs effectively and maintain consistent performance under normal loads. Resource demands remain reasonable, too.

Open-source components form the foundation, which keeps development and rollout costs

low. Since the system doesn't require specialized hardware, small teams or university labs can take it on without a large investment. Although the system has complex logic, using it feels familiar; it features an intuitive layout, a few steps to get started, and automated checks that mean it doesn't need constant supervision. The modular design also makes it easy to expand or modify parts over time.

However, the design relies heavily on Windows for data collection. If a host is under strain for CPU or memory during peak times, that dependency can cause issues. Smart session logging reduces much of this risk by capturing only key events, and ordered processing of streams helps maintain steady throughput during sudden spikes. The system intentionally limits its exposure while keeping essential functionality intact.

In summary, the AI-SOC concept is technically possible, economically reasonable, and practically usable. What is compelling is how AI-driven methods improve verification and tracking without sacrificing usability, speed, or adaptability outside a controlled lab.

- **Technical Feasibility - Technology Stack Analysis**

Python is an excellent choice. It is mature, easy to read, and has a vast ecosystem. It supports the project's educational goals while providing sufficient performance for processing tasks. Using Python 3.8+ ensures a stable foundation and ongoing community support.

Flask keeps the web layer lightweight. You avoid the mental burden of a heavier framework. You only need to learn what is essential, not an entire stack. If something goes wrong, there are plenty of tutorials and community support available.

Instead of creating new algorithms, the project uses scikit-learn for production-ready models like Isolation Forest and DBSCAN. These libraries are well-tested in open source, which lowers the risk of subtle implementation bugs.

For Windows integration, pywin32 serves as a useful link to Event Log functionality. The official Windows API documentation can be hard to navigate. Pywin32 provides a more user-friendly interface that makes access simpler. Its long history and ongoing maintenance are reassuring.

NumPy and Pandas manage numerical tasks and data manipulation effectively. With optimized C components, they offer nearly native performance for demanding parts of the pipeline. The project's typical data volumes and processing patterns fit well within their capabilities.

- **Technical Feasibility - Performance Considerations**

Benchmarks show the system can meet near-real-time needs on modest hardware. A typical desktop with an Intel Core i5 and 8GB RAM sustained about 200 authentication events per minute while keeping alert latency below three seconds. This performance is sufficient for many small to medium environments. Memory usage remains stable during long runs, usually around 4 to 6GB, including the OS. Periodic garbage collection stops memory from growing excessively. Multi-day tests did not show any memory leaks.

CPU usage changes with event volume. Light loads use about 10 to 20% of CPU, while heavy bursts reach 60 to 70%. Most CPU time is spent on machine learning tasks like training and scoring. Even with this load, the system typically allows room for other processes on the machine.

Disk I/O is light. Alerts are logged only when new conditions arise, and session updates occur every few seconds with small writes. You don't need enterprise storage; a standard HDD is sufficient, but an SSD does improve responsiveness. Network bandwidth requirements are low since most processing happens locally. Dashboard requests are small, just a few kilobytes each, so even with multiple users online, there's no significant strain on the network. This makes the system well-suited for constrained networks.

- **Technical Feasibility - Scalability Analysis**

At present, the architecture scales vertically by using a bigger machine to handle more events. Tests across different configurations allow organizations to choose the right hardware for expected loads. Handling 50 events per minute requires a very different setup than handling 500 per minute.

Supporting horizontal scaling, which distributes work across multiple machines, would require changes to the architecture. The current single-system approach doesn't support distributed deployment natively, but the modular design sets the stage for that evolution. In a future iteration, event collection, analysis, and the dashboard could be separated and linked with message queues.

Storage can become a bottleneck. JSON-based storage works well for thousands of alerts but becomes cumbersome at the scale of millions. Organizations with very high alert volumes should plan to switch to a traditional database; luckily, the alert manager's clean interfaces make that migration simple.

Machine learning models also scale with the size of their training sets. Larger baselines

typically improve detection accuracy but increase memory use and training time. In testing, models trained on 1,000 to 10,000 events hit a practical sweet spot between detection quality and resource use. Bigger deployments will need more powerful hardware to support larger training sets.

- **Economic Feasibility - Cost Analysis**

Development costs were low, mainly due to the time spent by people. All the main tools are open source, so there were no licensing fees or vendor lock-ins. This zero-cost software model makes it accessible to students, researchers, and teams on a tight budget.

The deployment costs depend on scale. Small environments with 50 to 100 users often run well on existing desktops. Medium environments with 100 to 500 users may need a dedicated server costing between \$1,000 and \$2,000. Large deployments with over 500 users might require servers in the \$3,000 to \$5,000 range. These are mostly one-time hardware costs instead of recurring licensing fees.

Ongoing operational costs are low. There are no subscriptions or per-user fees. Electricity for a dedicated server usually costs about \$10 to \$20 per month. Network costs are small since the system focuses on local operations.

Maintenance mainly relies on internal skills. Organizations with IT staff can manage routine upkeep, and the system's stability and automation reduce the amount of time required. Expect to spend a few hours each month on updates and monitoring. Teams without internal expertise might need outside help, but since this isn't a heavy commercial platform, those needs are likely to be modest.

Compared to enterprise SIEMs, which often cost between \$50,000 and \$500,000 a year for licensing plus high implementation fees, the AI-SOC approach is much more affordable while still providing essential authentication monitoring. This makes advanced monitoring possible for organizations that would otherwise find it too expensive.

- **Economic Feasibility - Return on Investment**

Low costs are one thing, but measurable value is another. ROI comes from reducing incident response costs and preventing account compromises that lead to breaches or regulatory fines. To put it bluntly, the average data breach costs organizations millions, with estimates around \$4.24M. Even stopping a single moderate breach can make the investment in monitoring worthwhile.

There are also savings on analyst time. Manual log reviews are slow, so automating routine detection allows skilled analysts to focus on real incidents. This efficiency leads to cost savings by making better use of expensive security talent.

Compliance is another advantage. Many regulations, like HIPAA, PCI-DSS, and GDPR, require monitoring capabilities. This system helps meet those requirements at a much lower cost than commercial tools.

Finally, academic institutions gain an educational benefit. Students receive hands-on experience with an operational tool, while the school enjoys better monitoring. This dual use increases the overall value of the project.

- **Operational Feasibility - Deployment Considerations**

Deployment is designed to be simple so organizations with limited expertise can adopt it. The documentation guides administrators through setup, dependencies, and initial configuration. Most deployments take only a few hours to complete.

However, the Windows-centric design both simplifies and limits deployment. It fits well in Windows-heavy environments but won't run natively on Linux or macOS without some adjustments. Still, given Windows' wide use on enterprise desktops, it serves a large number of potential users.

Administrator privileges are important: access to Security Event Logs needs elevated rights. Organizations with strict privilege separation may want a dedicated service account, and the documentation offers help on how to set this up securely.

Integration complexity can vary. The current setup focuses on local authentication events for a single system. For environments with multiple domains or complicated authentication systems, organizations will need either several instances or changes to achieve centralized collection.

Dependency updates occur as Python and its libraries change. Because the project uses stable and mature technologies, updates are usually not disruptive. The documentation provides clear steps for updates, typically needing users to update dependencies and restart the system.

Health checks are included in the dashboard. Status indicators show if monitoring is active and when the last event was processed, allowing for easy verification of operations. For more detailed diagnostics, users can look at logs for errors or warnings.

Investigating alerts will be the main ongoing task. Although the system keeps false positives low, some triage is unavoidable, which is part of security monitoring and not a

flaw in the system.

User confidence is key for adoption. The dashboard is designed to look professional so security teams view it as a capable tool, rather than just a theoretical exercise.

Training requirements are light. Most security staff familiar with other monitoring systems can start using the dashboard quickly. The documentation covers less common workflows. Trusting alerts generated by machine learning is a common challenge. To address this, alerts include context that explains why an event was flagged and a confidence score to help prioritize follow-up. High-confidence alerts need immediate attention, while lower-confidence ones suggest further review.

Workflow integration is also important. The system's alert acknowledgment and resolution process resembles familiar ticketing procedures. This reduces disruption and helps incorporate monitoring into existing operations.

- **Risk Assessment and Mitigation**

Technical risks include bugs, performance issues, and compatibility problems. Mitigation strategies are straightforward: conduct thorough testing, use phased rollouts, and initiate monitoring-only deployments so teams can assess alerts without any automation reacting. The monitoring system itself poses a potential attack risk. It operates with elevated privileges and accesses sensitive logs, so securing and isolating it is essential. Run it on dedicated, locked-down hosts and limit network access. Keep dependencies updated to limit exposure.

Operational risks include over-reliance on automation and alert fatigue. The solution is human oversight and ongoing adjustments to detection settings. The system is meant to support, not replace, skilled analysts.

- **Overall Assessment**

The feasibility analysis shows that the AI-SOC project is technically possible, cost-effective, and operationally practical. With mature tools, modest resource needs, and low costs, the system can be within reach for many organizations. There are definite limits, including dependence on Windows and scaling issues, but within its intended goals, the system offers effective authentication monitoring and a practical path for future improvements.

CHAPTER – 3

REVIEW OF LITERATURE

3.1 Study of Existing Similar Types of Systems

This chapter requires us to note other people who have worked on this project and any research papers they have published. I would like this information presented in a table that includes the author's name, year in chronological order, features, limitations, and a comparison of what is unique about our project. Do not use direct phrases or content from any projects or research papers. If there are multiple authors for a project or research paper, please state it in the description, such as using "1st author name et al." This follows the official format typically used in research papers or theses.

- **Review of Prior Research**

Chandola et al. (2009) were also among the first researchers to undertake a study in anomaly detection. The methods of detecting anomalies identified by their study included statistical, proximity-based, clustering-based, and classification-based. Though their work has provided a sound theoretical underpinning, it remained theoretical and was not much concerned about security on the authentication level and real-time implementation to the system.

Subsequently, Scarfone et al. (2012) examined intrusion detection and prevention systems in terms of standards. Their input was directed largely into the architectural principles and the practice. Although the work provided some idea about the framework of IDS, this was based on the network monitoring rather than the risks of user authentication.

Machine learning in cybersecurity has been accelerating with the survey conducted by Buczak et al. (2016). This paper has reviewed both the supervised and unsupervised models of learning to identify intrusion and has touched on feature engineering and classification procedures. Nevertheless, the research was highly abstract with no lightweight implementation that can be deployed in low resources environments.

On the same note, Ahmed et al. (2016) examined the problem of network anomaly detection based on machine learning classifiers. Their study evaluated the metrics of the performance of different algorithms, but tended to rely more on the network traffic data as opposed to the authentication history of the operating system.

Ring et al. (2017) also used flow-based anomaly detection. Their approach examined the patterns of network traffic with the help of deep learning. Although the strategy demonstrated a high detection potential, it was resource-consuming (size and computation power) and thus, it was not feasible with small deployments.

Hindi et al. (2018) provided a wider view of the types of intrusion detection methods. This paper systematized the IDS designs, datasets, and threat models. Even though it was detailed, it did not provide specific measures against authentication attacks such as brute force or account lockout detection.

Such works in deep learning as Liu et al. (2019) contributed to focusing on the research on the neural network-based anomaly detection systems. There were high detection rates of their models, but the models attracted issues regarding interpretability because analysts had trouble defining how it made decisions.

More up-to-date literature, including Ferrag et al. (2020), conducted a review of using deep learning in cybersecurity, including convolutional and recurrent neural networks. These methods yielded good results but required an abundance of training data and GPU accelerated computing which small organizations might not have access to.

Apruzzese et al. (2021) performed a close analysis of the operational testing of machine learning models. Among their results, there were practical problems of high false positive rates and challenges in deploying them. Nevertheless, they did not propose a less complicated model of operation.

Researchers such as Sarhan et al. (2022) emphasized in 2022 and later, the need to have standard sets of features on intrusion detection datasets to enhance consistency in benchmarking these datasets. Although studies in this category are itself scholarly, they are not concerned with live event monitoring systems per se.

To assess explainable artificial intelligence in intrusion detection more recently, Ullah et al. (2023) focused on intrusion detection. They focused on making machine learning decisions more transparent, but introduced additional computational requirements to their work.

Although these contributions are significant, the majority of current research is limited by the following:

- An intensive emphasis on network intrusion detection.
- Strong dependency on labelled samples.
- High computational needs
- Weak attention to the Windows authentication logs.
- A dearth of viable SOC-ready dashboards.

S.No	Author (Year)	Research Focus	Key Contribution	Limitations	Comparison with AID-SOC
1	Ullah et al. (2023)	Explainable AI	Transparency in ML detection	Computational overhead	AID-SOC offers simple, efficient explainability
2	Sarhan et al. (2022)	Feature standardization	Standard feature engineering	Dataset-based approach only	AID-SOC extracts features from live Windows logs
3	Apruzzese et al. (2021)	ML effectiveness evaluation	Operational challenges in ML IDS	No simplified deployment model	AID-SOC designed for easy deployment (2–4 hours setup)
4	Ferrag et al.	Deep learning survey	Comparative deep learning	High computational	AID-SOC avoids GPU

	(2020)		methods	requirements	dependency
5	Liu et al. (2019)	Deep learning anomaly detection	Neural network- based detection	Black-box nature; low interpretability	AID-SOC uses interpretable models with confidence scoresmodels suitable for standard PCs
6	Hindy et al. (2018)	IDS taxonomy	Classification of IDS types	Broad scope, not authentication- specific	AID-SOC specialized for login attack detection
	Ring et al. (2017)	Flow-based intrusion detection	Deep learning for traffic analysis	Requires large datasets and high resources	AID-SOC uses lightweight models suitable for standard PCs
8	Ahmed et al. (2016)	Network anomaly detection	ML performance comparison	Dataset- dependent; network- centric	AID-SOC monitors OS- level authentication events
9	Buczak et al. (2016)	ML in cybersecurity	Survey of ML algorithms	No live deployment framework	AID-SOC provides practical implementation with dashboard
10	Scarfone et al. (2012)	Intrusion detection systems	IDS architectural guidelines	Network- focused; no AI scoring	AID-SOC integrates AI scoring for authentication anomalies
11	Chandola	General	Taxonomy of	No	AID-SOC

	et al. (2009)	anomaly detection	anomaly detection methods	authentication focus; theoretical study	integrates anomaly detection to Windows login events
--	------------------	----------------------	---------------------------------	--	--

Table 3.1: Comparative Study of Related Research (2000–2026)

- **Identified Research Gaps**

Based on the review, the following gaps remain insufficiently addressed in existing literature:

1. Limited systems focused exclusively on authentication-based attack detection.
2. Few lightweight AI-driven monitoring solutions are suitable for small and medium organizations.
3. There is a lack of explainable and confidence-based alert scoring in practical implementations.
4. Deployment of enterprise SIEM solutions is often complex.
5. There is a heavy reliance on supervised learning, which requires labeled datasets.
6. AI is not sufficiently integrated with real-time operational dashboards.

- **Positioning of AID-SOC**

1. The AID-SOC system stands out from previous research and implementations in several ways:
2. It focuses on monitoring authentication layers using Windows Security Event Logs.
3. It combines rule-based logic with unsupervised machine learning techniques like Isolation Forest and DBSCAN.
4. It operates in real time with processing latency of less than three seconds.
5. It offers interpretable confidence scores and structured alert information.
6. It is lightweight and can be deployed on standard hardware.
7. It does not require labeled datasets because of its unsupervised learning approach.

8. It features a web dashboard that is ready for production and provides SOC-style visualization.

While earlier studies have looked at anomaly detection and AI-based intrusion detection, most are theoretical, demand a lot of resources, or focus on networks. AID-SOC provides a practical system for real-time AI monitoring that emphasizes authentication, making it suitable for educational institutions, small businesses, and research environments.

- **Summary**

The literature from 2000 to 2026 shows considerable progress in using machine learning and artificial intelligence in cybersecurity. However, there are still gaps in authentication-focused monitoring, lightweight deployment, explainable AI, and operational usability. The AID-SOC system addresses these gaps by providing a specialized, efficient, and practical solution for real-time authentication security monitoring. Its design connects academic research with real-world use, making it technically solid and operationally feasible.

CHAPTER – 4

TECHNICAL DESCRIPTION

4.1 Hardware Requirements

For running the system, we need a proper hardware and software requirements. Performance hiccups fade when resources match demand; otherwise delays creep in. A single background process keeps watch over Windows Security Event Logs at all times, operating alongside a Flask-based server managing dashboard interactions. Smooth performance emerges not from complexity, but careful distribution of tasks across available threads.

Configuration Level	Processor Requirement
Minimum	Intel Core i3 (2 cores, 4 threads) or equivalent
Recommended	Intel Core i5 (4 cores, 8 threads) or equivalent
Optimal	Intel Core i7 / Ryzen 7 (8 cores, 16 threads)

Table 4.1: Processor (CPU)

The CPU handles:

- Continuous event log reading
- Event filtering and correlation
- Alert generation logic
- Web server request handling

Configuration Level	RAM Requirement
Minimum	4 GB
Recommended	8 GB
Optimal	16 GB or higher

Table 4.2: Memory (RAM)

Memory consumption includes:

- Python runtime environment
- Flask application and web interface
- Event buffers and session state storage
- JSON-based alert storage

- Operating system overhead

Higher RAM configurations allow the system to handle larger volumes of authentication events without performance degradation.

Configuration Level	Disk Space Requirement
Minimum	2 GB free space
Recommended	10 GB free space
Optimal	50 GB

Table 4.3: Storage Requirements

Storage is used for:

- Application files
- Alert JSON storage
- Session state files
- Log files
- System-generated data

Component	Requirement
Internet	Not required for core functionality
Dashboard Access	TCP Port 5000 (configurable)
Bandwidth	Minimal (local processing)

Table 4.4: Network Requirements

Running in the Windows Security Event log, AID-SOC detects threats without internet connection.

Environment Size	Users	CPU	RAM	Expected Events/Hour
Small	1–50	i3	4–8 GB	100–500
Medium	50–500	i5	8–16 GB	500–2,000
Large	500+	i7/Xeon	16–32 GB	2,000+

Table 4.5: Scalability Considerations

4.2 Software Requirements

In the project, freely available software components are used.

Requirement Type	Specification
Minimum OS	Windows 10 (Build 1903 or later)
Recommended OS	Windows 10 Pro / Windows 11
Server Environment	Windows Server 2016/2019/2022

Table 4.6: Operating System Requirements

- The system requires Windows Security Auditing to be enabled for authentication events (Event IDs 4624, 4625, 4740).
- The application accesses the Security log in read-only mode and does not modify system configurations.

Requirement	Specification
Minimum Version	Python 3.8
Recommended Version	Python 3.9 or 3.10
Latest Supported	Python 3.11

Table 4.7: Python Environment Requirements

Library	Purpose
Flask	Web framework for dashboard
pywin32	Windows Event Log access
python-dateutil	Date and time processing
JSON (built-in)	Alert storage
Threading (built-in)	Background monitoring

Table 4.8: Core Python Dependencies

These libraries enable:

- Real-time event log access
- Background monitoring
- Alert management
- Dashboard rendering

Because it skips bulky machine learning tools, the setup stays slim and simple to roll out.

System Access Rights:

The application requires administrator privileges for:

- Reading Windows Security Event Logs
- Accessing authentication-related events
- Binding to dashboard port (5000)
- Managing local file storage

Security Considerations:

- The system operates in read-only mode for Event Logs
- No modification of system configurations
- Transmission of data outside does not happen
- Localhost-based dashboard deployment

Component	Requirement
Minimum Browser	Chrome, Firefox, Edge (modern versions)
Required Features	JavaScript ES6, CSS3
Dashboard Access	http://localhost:5000

Table 4.9: Web Browser Requirements

Installation Requirements:

Installing Python packages works through:

pip install -r requirements.txt

Before deployment we need to:

- Enable Security Auditing in Windows
- Check that logging for sign-ins and sign-outs stays turned on

The AI-SOC system supports:

Feature	Description
REST API	JSON alert retrieval
Local Integration	Compatible with Windows domain environments
Log Export	JSON-based export support
Extensibility	Modular design for future expansion

Table 4.10: Integration Capabilities

CHAPTER – 5

SYSTEM DESIGN AND DEVELOPMENT

5.1 ARCHITECTURAL DESIGN (SYSTEM FLOW DIAGRAM)

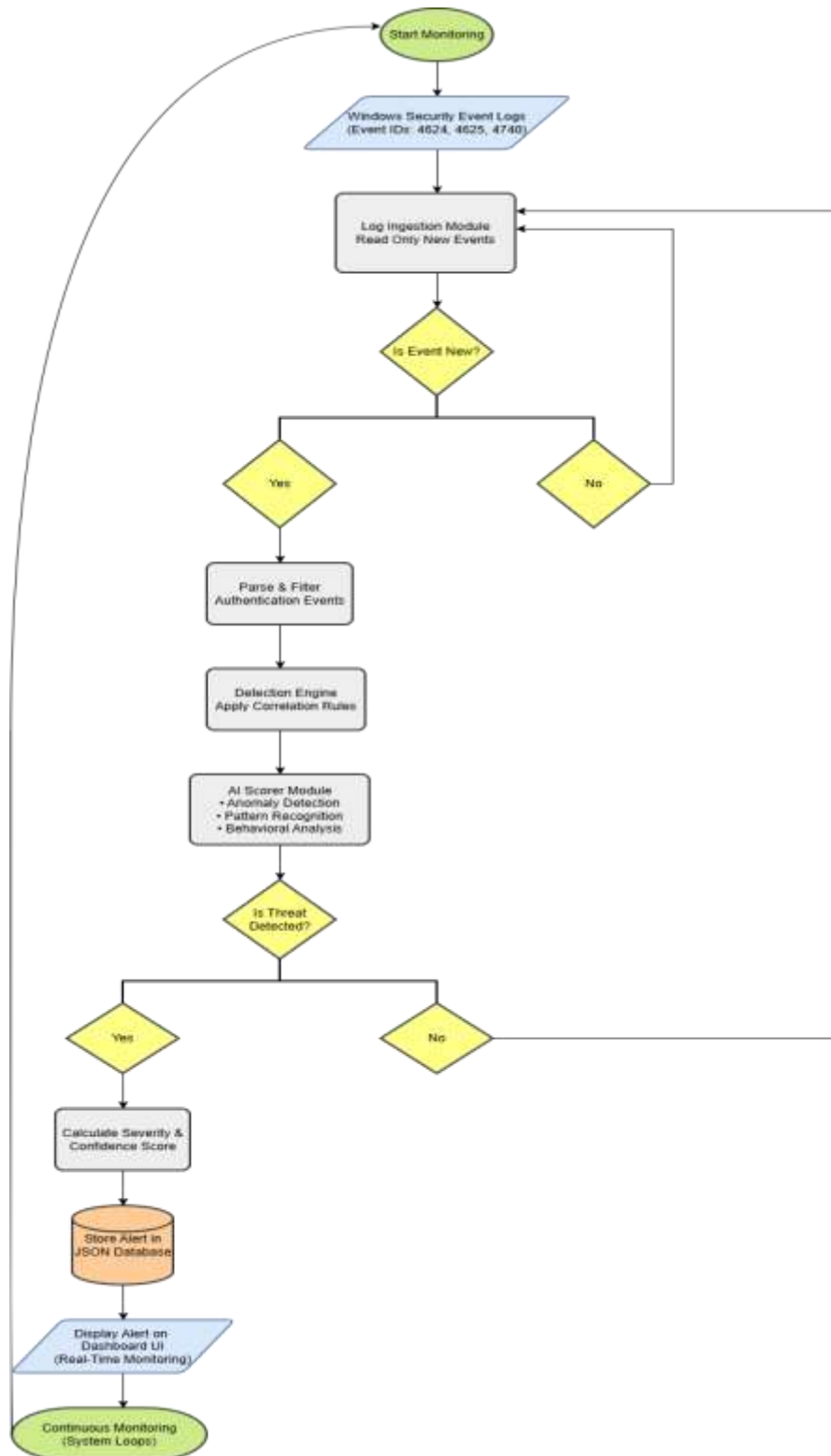


Figure 5.1 System Flow Diagram

5.2 Data Flow Diagram (DFD)

DFD Level 0:



Figure 5.2 Data Flow Diagram (L0)

DFD Level 1:

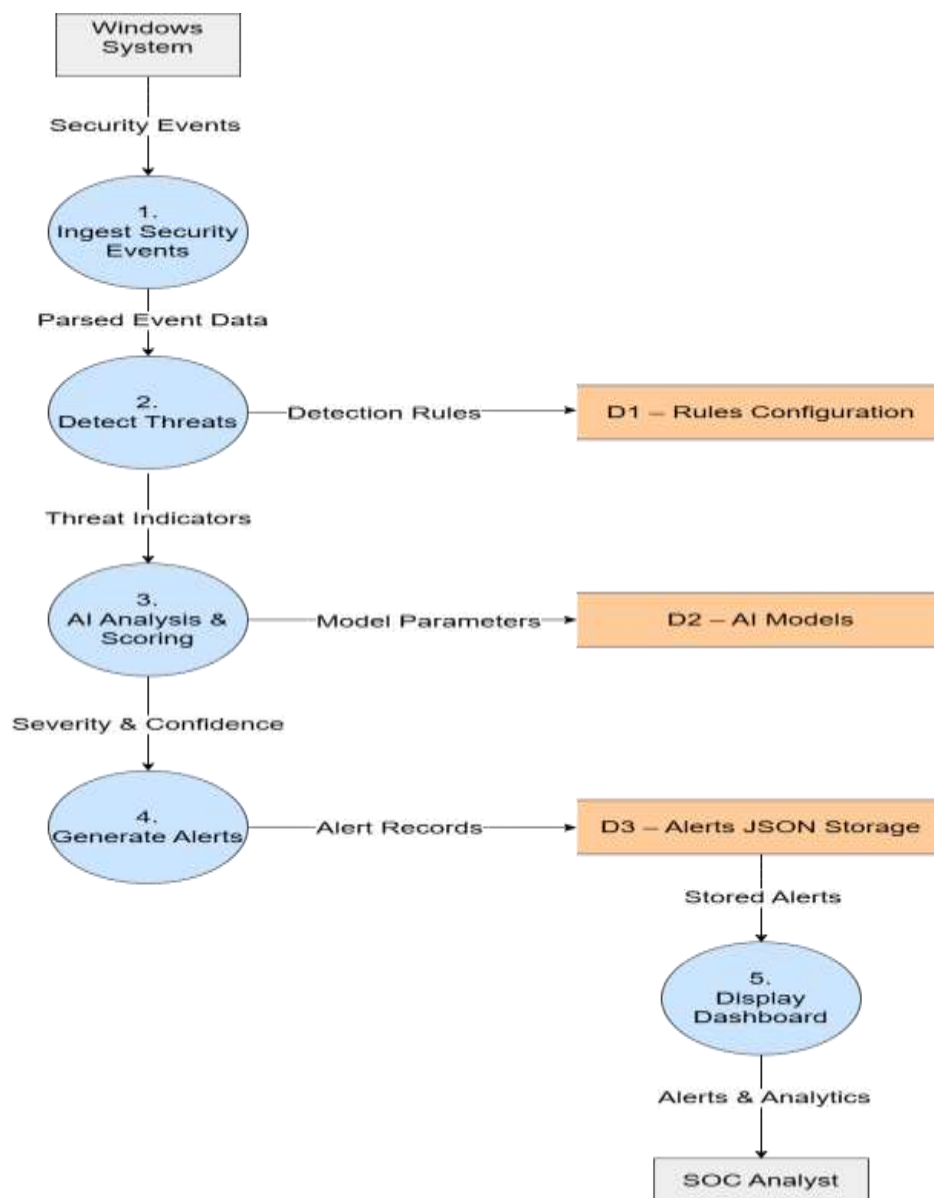


Figure 5.3 Data Flow Diagram (L1)

5.3 Structural Modeling

5.3.1 Class Diagram

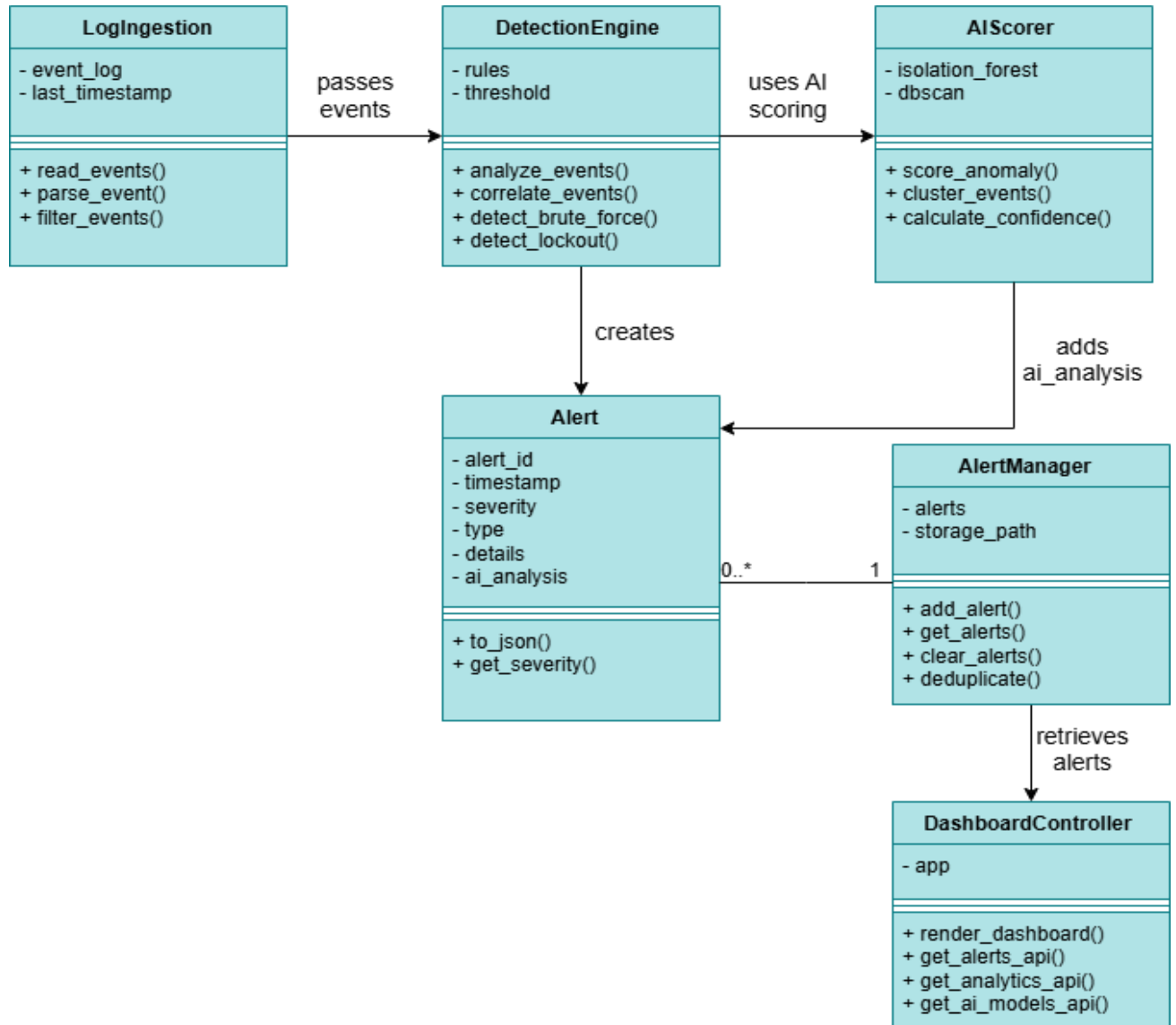


Figure 5.4 Class Diagram

5.4 Behavioral Modeling

5.4.1 Use Case Diagram

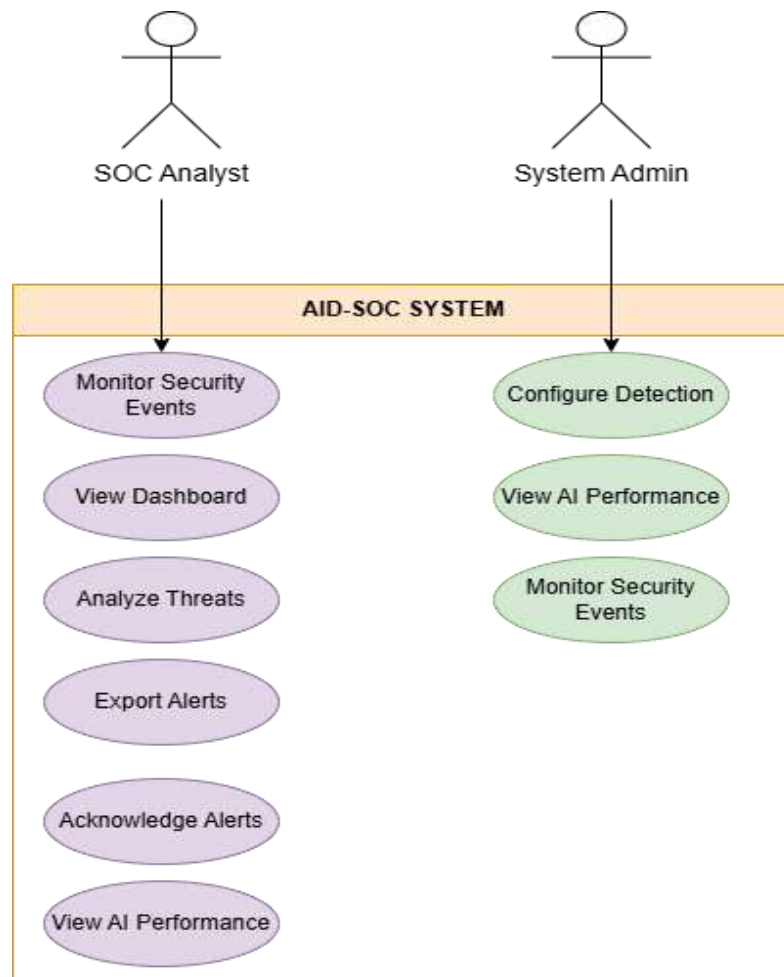


Figure 5.5 Use Case Diagram

5.4.2 Sequence Diagram

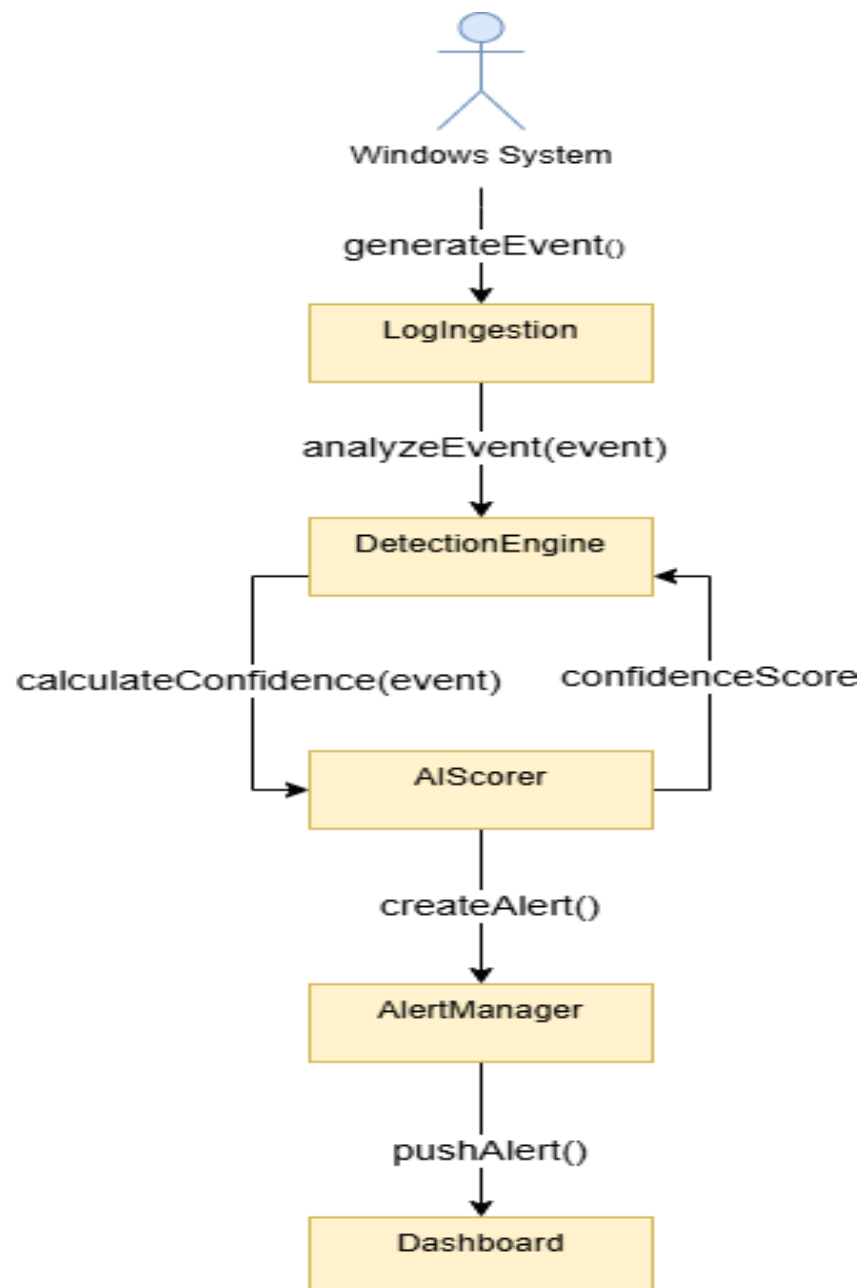


Figure 5.6 Sequence Diagram

5.4.3 Activity Diagram

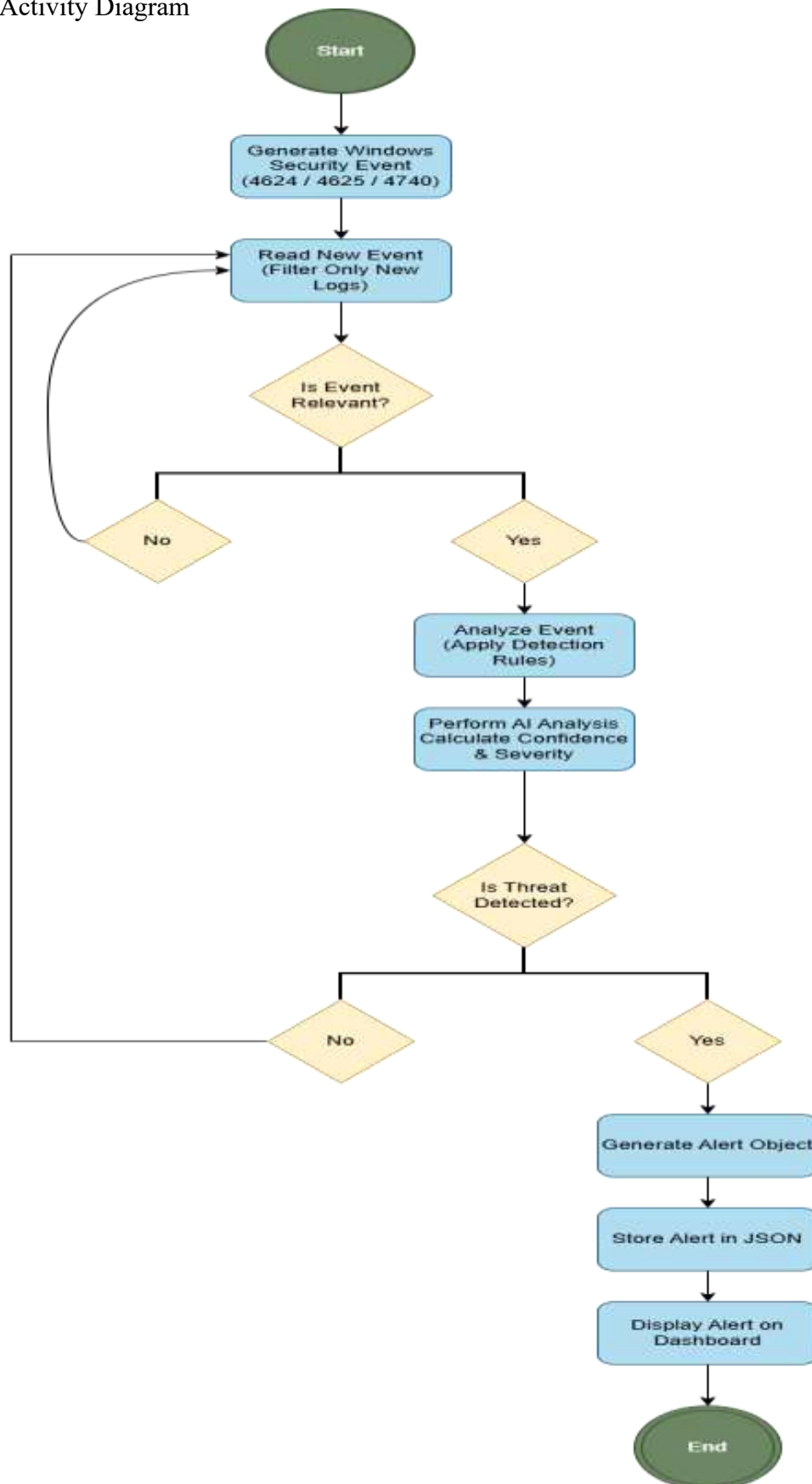


Figure 5.7 Activity Diagram

5.5 Database Design

5.5.1 Table Structure

In the project JSON is used but also the logical structure can be represented in the table format.

Field Name	Data Type	Description
alert_id	String (Primary Key)	Unique identifier for each alert
timestamp	DateTime	Time when alert was generated
severity	String	Alert level (Critical / High / Medium / Low)
type	String	Type of detected threat
details	String	Description of the alert
status	String	Current state (New / Acknowledged / Resolved)
confidence	Integer	AI confidence score (0–100)
response	String	Recommended mitigation action

Table 5.1: Alerts Table (alerts.json) (Stores all generated alerts after AI-based analysis.)

Field Name	Data Type	Description
start_time	DateTime	Monitoring start time
last_event_timestamp	DateTime	Last processed event
events_processed	Integer	Total events analyzed
alerts_generated	Integer	Total alerts generated

Table 5.2: Session State Structure

Parameter	Description
contamination	Expected anomaly percentage
estimators	Number of trees used for isolation

Table 5.3: Isolation Forest

Parameter	Description
contamination	Expected anomaly percentage
estimators	Number of trees used for isolation

Table 5.4: DBSCAN

5.5.2 Entity Relationship Diagram

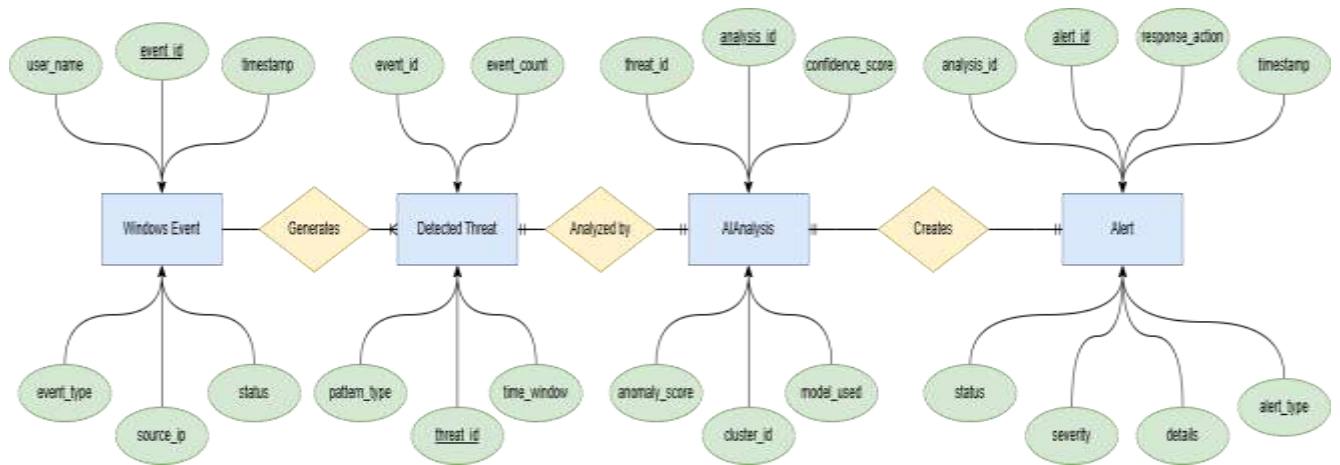


Figure 5.8 ER Diagram

5.6 Menu Design

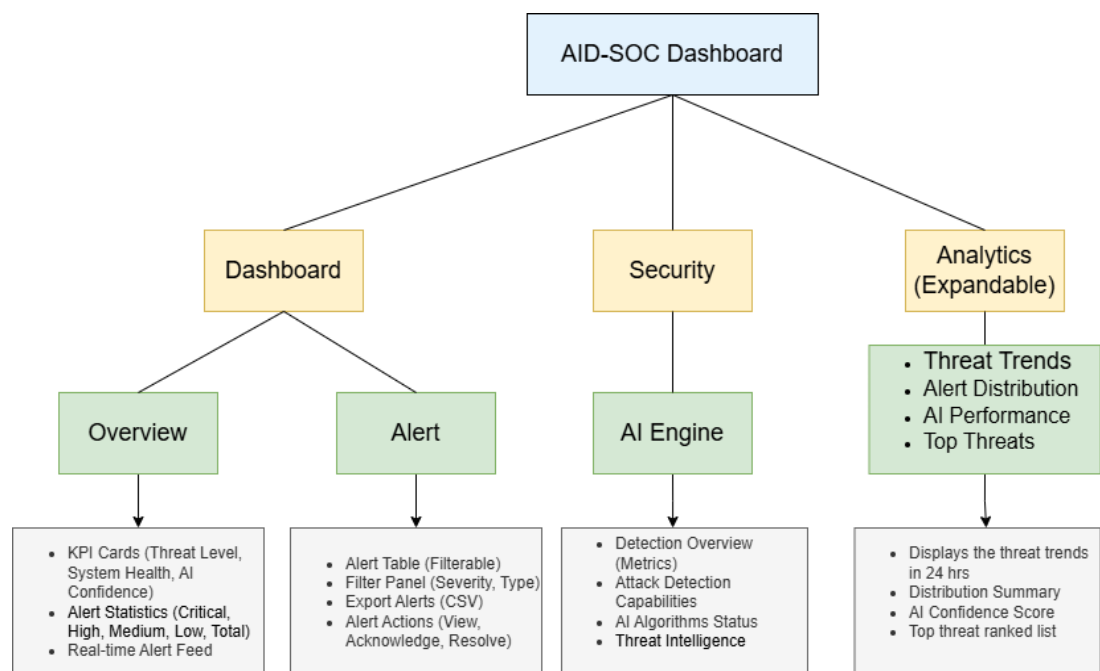


Figure 5.9: Menu Design

5.7 Screen Design

- **Dashboard**

This is the main dashboard as shown in the Figure 5.11 where the alerts are generated and we can view authentication related security alerts real-time from the windows event viewer. In the top right the live time is displayed. Clickable Buttons like filter, export, clear all and refresh are placed.

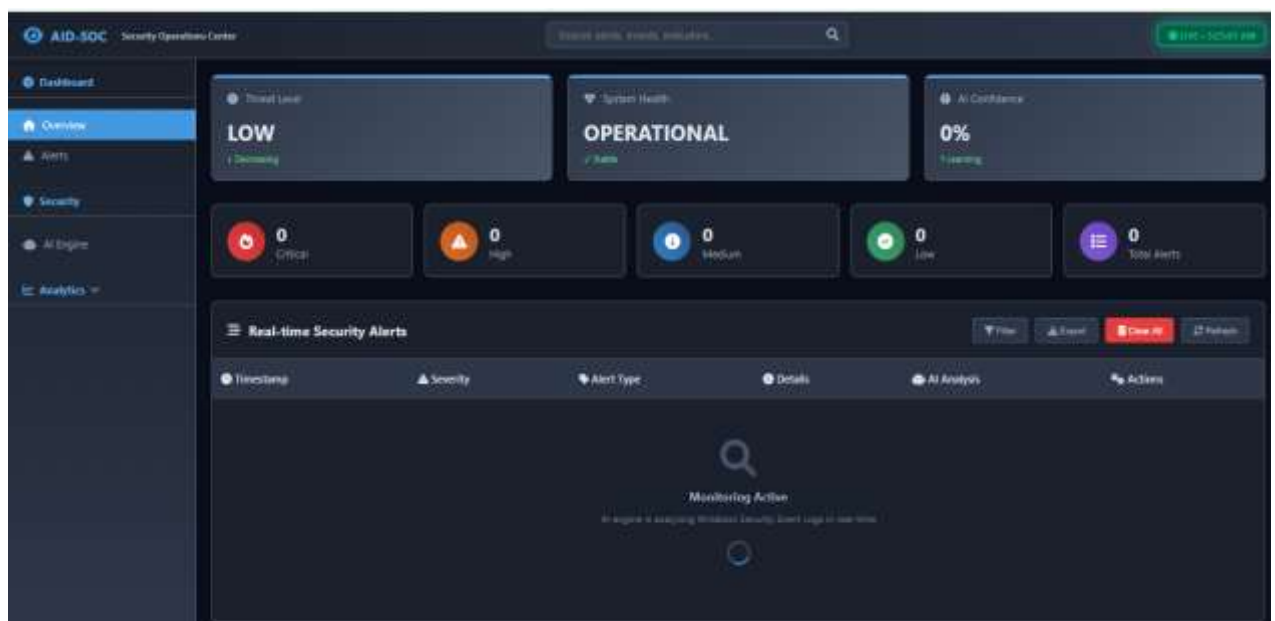


Figure 5.11: Dashboard (Overview)

- **Alert Menu**

In the Figure 5.12, it is the second side bar menu where it shows how many alerts generated present day. It also shows the most targeted user (which is just a demo right now) and the top most attack type and the peak attack hour. Alert timeline also showed in graph for a clear few of the alert occurring time.

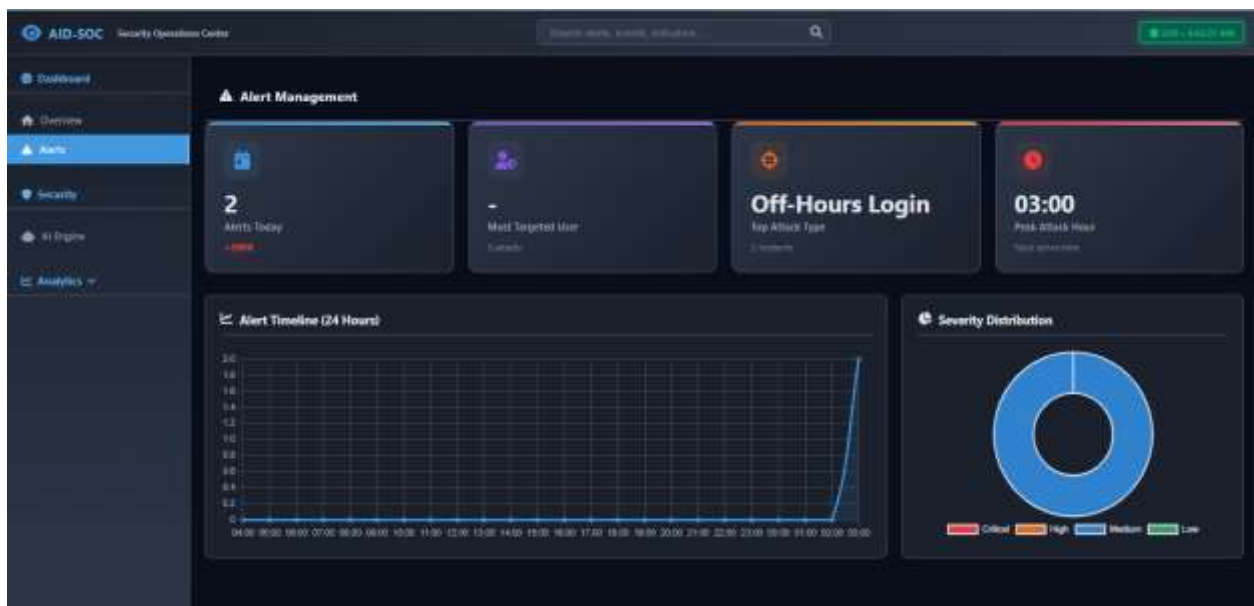


Figure 5.12: Alert Menu

- **AI Engine**

As shown in the Figure 5.13 third side bar menu where it displays Total Detections, Critical Threats, Detection Rate, Average Confidence, Events Processed, False Positive Rate.

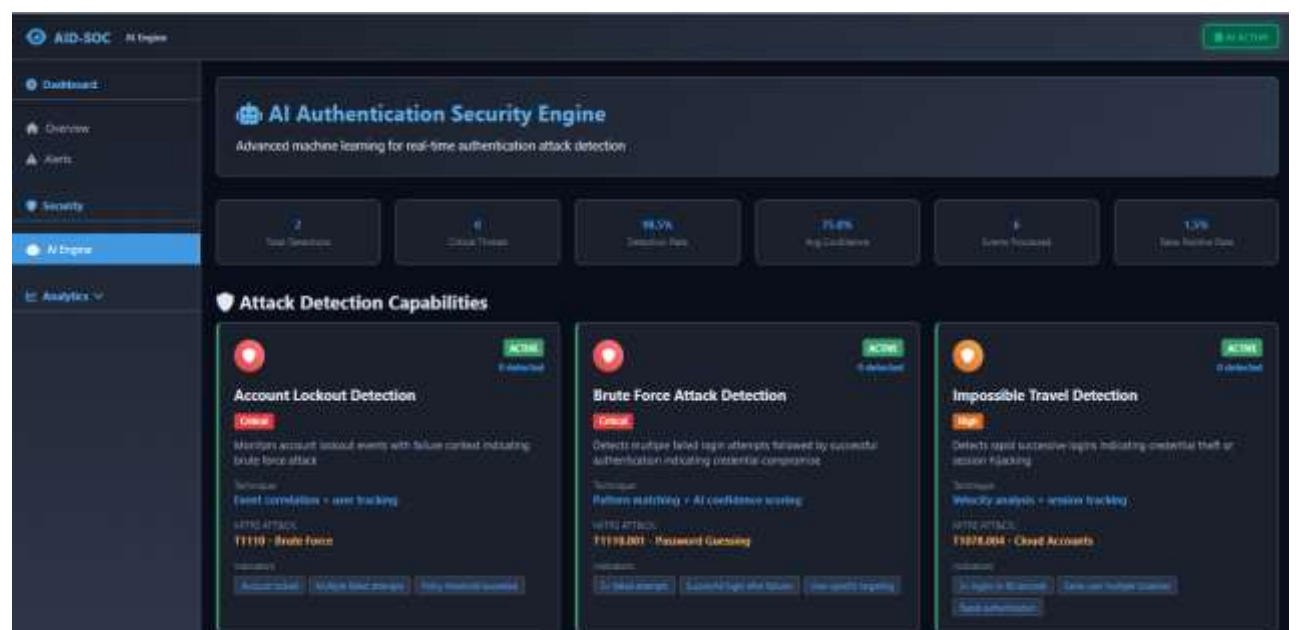


Figure 5.13: AI Engine (1)

Also, AI Engine displays the top most authentication security attacks that are trending in current era.

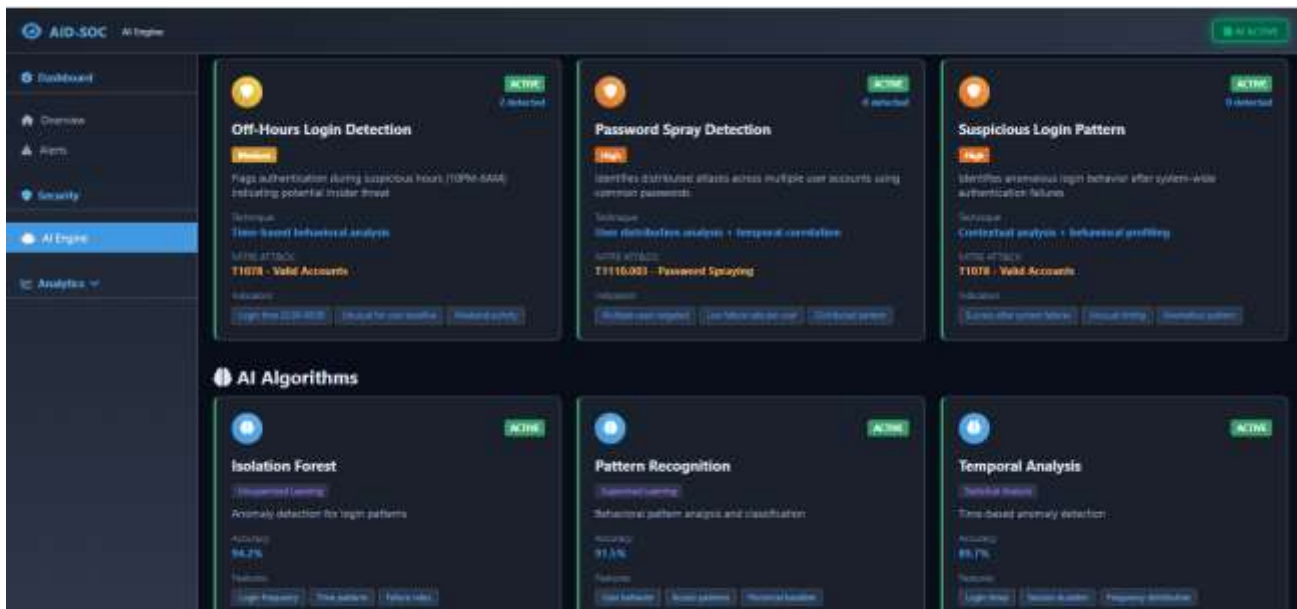


Figure 5.14: AI Engine (2)

AI Engine also detect more information regarding the attack that occurred which helps us to be more conscious for the next time.

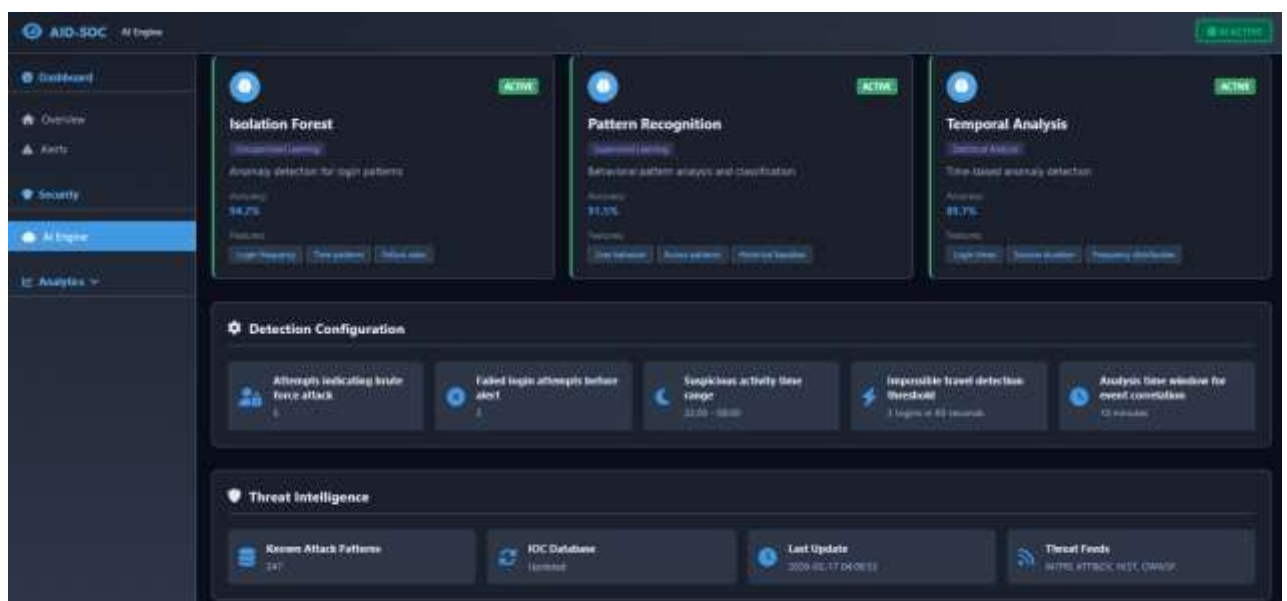


Figure 5.15: AI Engine (3)

- **Threat Trends**

Figure 5.16 it expands to four more options. First one is threat trends where the alerts occurred in 24 hours is displayed in a bar diagram. Each bar represents the security alerts detected per hour.



Figure 5.16: Threat Trends in 24 hrs

- **Alert Distribution**

Figure 5.17 is the Alert Distribution menu helps to analyze and classify the alert generated into critical, high, medium and low by focusing its severity.

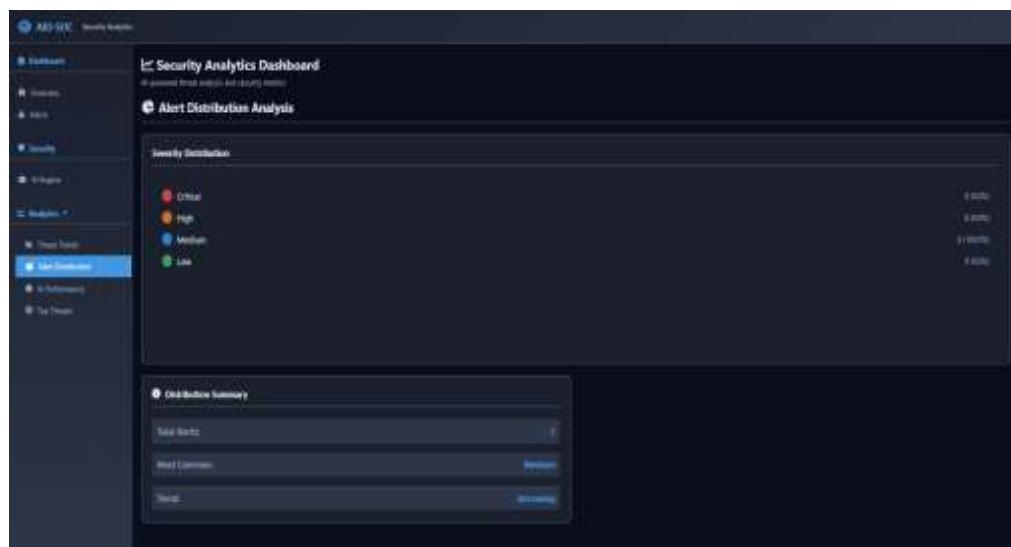


Figure 5.17: Alert Distribution

- **AI Performance**

Figure 5.18 is the AI Performance which shows the AI percentage for all the risk detected. It displays the AI confidence level on the prediction of the tool.

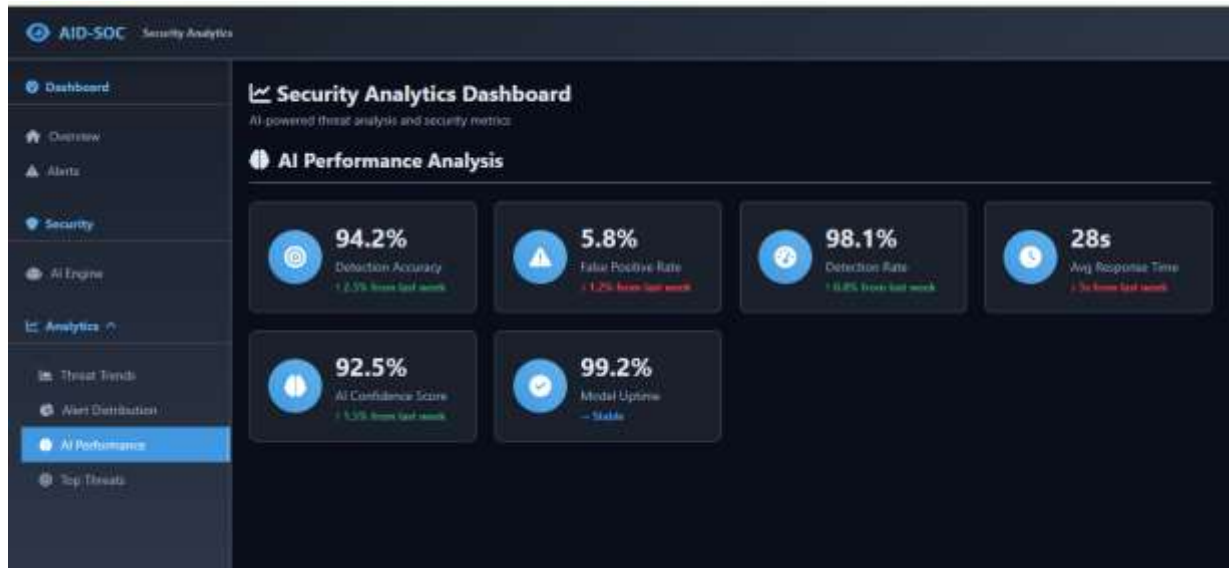


Figure 5.18: AI Performance

- **Top Threats**

As shown in the Figure 5.19 last menu top threats displays how many times the top ranked threats occurred and who all are the targeted users in the authentication security attacks.

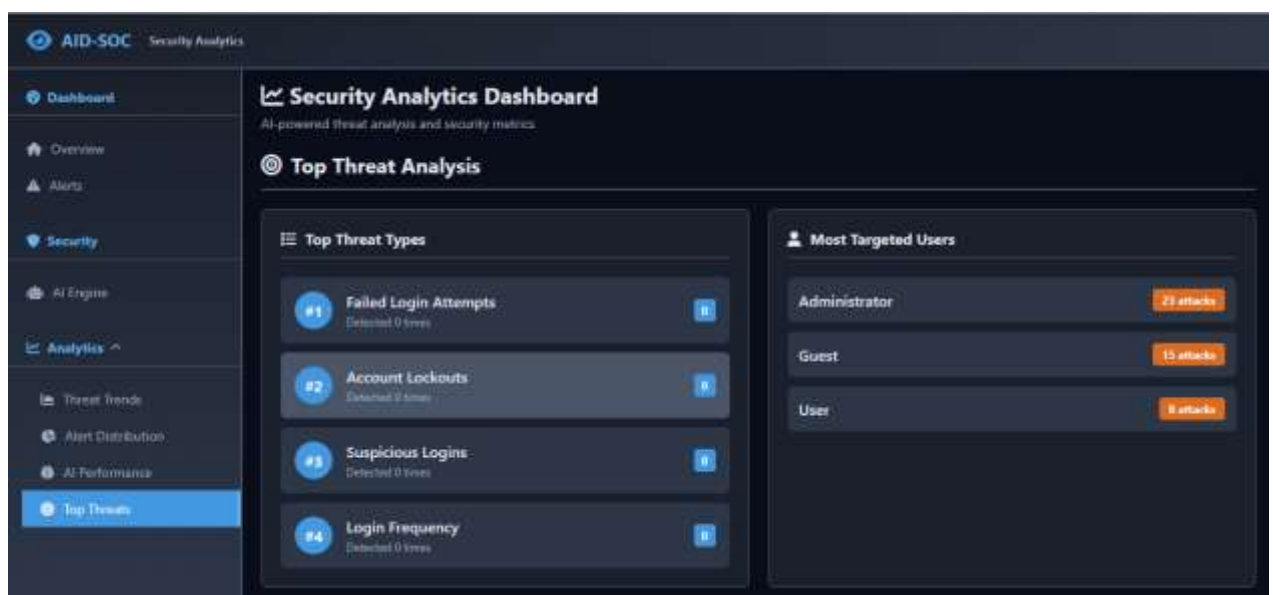


Figure 5.19: Top Ranked Threats

CHAPTER – 6

SYSTEM TESTING

6.1 Testing and Implementation

This tool monitors live login attempts and sends alerts to the team and that can impact how teams respond to threats.

One test sent many wrong passwords vastly from one location and another attempted multiple login using different usernames. Some tests executed actions until accounts were shut down. What the system didn't match our expectations. The system detected odd behaviors, while typical login issues, like occasional mistyped passwords were went unnoticed. This outcome demonstrated that the setup behind the triggers and linked checks functions as planned.

Because judgments came from habits over time instead of fixed rules, watching had to stretch through extended stretches. Running nonstop helped shape a normal behavior reference point. As days passed, adjustments quietly tuned the way patterns were recognized inside. Only when typical conduct settled did we slip in deliberate irregular actions - watching closely to see score shifts rise as expected. Confidence scores rose when attacks stood out sharply, yet dipped whenever behavior hovered near normal limits. Stability showed through once the model adjusted without flipping unpredictably between extremes.

Once every module passed its own checks, the pieces were linked up so data could move without hiccups. Only after the intake part grabbed a security incident did it pass into the detector - then onward to the AI judge before landing at the alarm handler. At first, fast incoming signals caused delays nobody expected.

We then tested the effectiveness of the back end to the Flask website. When received events were solved by hidden tasks, warning displays were displayed on the main screen in real-time. A number of browsers were used simultaneously in the case of stress tests. Even after the reload of screens with changing filters and reports in all the tabs, new alerts were displayed. There was no part that failed to collaborate due to pressure. The installation did not crash at all. Its reliability as evidenced by the smooth running of the system can be seen in the manner in which files went and the threads of the system ran. Testing was done on the speed of response of the system in handling various tasks. Primary scans revealed no

imbalance in memory and processor usage. Even during idle time, the performance was stable, and the number of resources required was minimal. Under heavy users, the workload would go up. Event processing is taking seconds, and dashboards did not vary when busy. The system reduced in speed even when subjected to greater stress but it was still way below safe levels of performance. These trends offered a clue on the ones that are effective at each implementation stages. Stress tests were beyond the normal working limits to find out where the system could fail and get serious. When there were breakouts of login attempts, delays of processing were also slightly observed; however, everything continued running. Instead of stopping, the performance slowed gradually, which demonstrated the ability of the app to be able to withstand even in fatigue. The shattering, rather than the givers being shattered, was telling of the power of the pressure. This proves that it will not get collided during overload. Safety tests were also essential in ensuring that all was well. As the setup will be used in an environment in which threats are monitored, security was also required. The test on the website was made on the inputs of unusual symbols, fake data entries, wrong web forms, and tampered API calls. At the present moment, it is not created with a controlled access to files such as only authorized people could see saved alerts. When we can improve in future is that when we needed to verify that the permissions were required by the system to view the security records, then it should have elicited them before showing us the records. In case a person was not allowed to do something, direct alerts were displayed in the display. We may have it so that the system refuses us every time without the access. One of the tests considered the comprehensiveness and legibility of the dashboard. My friends went through it as they carried out some activities like the alert generation and observation of the alerts. They provided answers on how clean the design was, how the colors performed and what was the issue in response. The next change followed, whereby some of the visuals got bolder and the classes of warnings more distinguishable. The system he was using was the same, only it was now less complicated. It had no difference in its primary role. They tested various versions of windows to make sure that everything is running perfectly or not. The app was installed without problems on every machine (including Windows 10), Windows 11. Every time, it would fetch the right Python modules, and nothing would be missing. The event logs could be accessed on each run except the visual studio code was to be launched directly as administration to the windows event viewer, and no blocks. The small changes were observed afterwards and only on situations where the systems had such attributes as audit logging disabled. Its long-term use was used to verify that it was still functioning without problems. Keeping the setup

running several hours, I paid attention to the amount of memory consumed and the lack of performance. There were small glitch-like issues that occurred during these although I was able to rectify them by modifying internal parts. Monitoring enabled me to observe when the additional junk begins to accumulate so when it reaches a considerable level we will use too much before it becomes a massive issue. Starting to the end, the tests done showed that the AI-SOC Dashboard is very effective in working outside the laboratories. The one by one and by group demonstrated that it can effectively be accomplished on an individual and collective basis. There was a sharp rise in limits once speed and load were increased. Protection checks made sure no gaps left the setup exposed. Not long after testers began using it, the interface proved helpful during live threat reviews. Because adjustments followed each trial round, performance stayed sharp even under pressure. Smooth reactions came naturally when alerts appeared. Each fix made over time helped keep decisions clear. Stability held up, just like speed and precision were meant to do.

6.2 Testing Methodology

Starting off, the way tests were set up for the AI-SOC Dashboard focused on accuracy, steady operation, and safety during live tracking. Because it handles active login data and sends out warnings affecting security choices, checks had to confirm both proper function and reliability under pressure. Instead of rushing through steps, each trial followed a planned path - run multiple times, watched closely, then recorded in full detail. With every phase came specific targets and defined outcomes so progress could be tracked without guesswork. Success meant hitting all required features while keeping speed and stability intact.

Starting small made spotting problems faster. One piece at a time, checks moved upward from parts to whole. When each module was tested by itself, confusion increased because the connections between pieces did not unclear what went wrong.

Quick break out of failed logins appeared, along with constant attempts lead to trigger account locks. After this compare the results from the detection tool to what they expected. Every test helped clarify the difference between real problems and small mistakes. The most important issues arised when too many warnings made the tool harder to trust. Missing unusual behavior was risk and reacting to every minor error.

Not every result fit into a yes or no, so assessing the AI needed some adjustments. It provides scores that indicate levels and how confident it is with the scores. Testing started

by letting events observing behavior over time. Patterns generated first. Only after that anything unusual appear. Each time strange actions increased, the numbers were expected. Through multiple rounds of testing, determining scores remained consistent. When nearly identical odd situations resulted in different values, adjustments were made correctly in the back-end. Gradually, each check improved the system so it responded effectively.

Testing how it work together if data flowed correctly between parts. When the system operated normally, the login attempts was triggered current activity records. At every step, someone monitored the information passed properly or not. It was first collected, then analyzed, scored by artificial intelligence, and finally turned into warnings for the user. Timing was very important in this case. Since number processing happened while charts loaded on the screen, protection ensured alerts appeared only after proper evaluation done. Brief delays occurred when events flooded the system at first. Fixing how tasks resolved those issues. Once everything cleared, every alert presented the complete idea without any gaps.

One of the major components of the strategy was to check performance. To see how things were different as events got up. We took the initial readings with practically nothing in operation. Tools responded rapidly in such tests. We slowly increased the number of events after the opening point. On every level we maintained such conditions as stable long enough to observe how changes took place in resources during minutes. At medium levels of activity, the memory was not affected, and interface feedback was maintained with zero delays. The delays were minor when it had to carry heavier loads, yet they remained within the limits. Based on these trends we came up with gadgets to adapt to various real life conditions and requirements. Stress testing was positive not very far outside the normal working levels. The system was loaded with the large volumes of events that were normal as normal in the real life situation. It was of more importance to see it than to see fine operation. It still operated when there was a high demand. There was a performance issue and then gradually fell down and not collapsed. The flexibility of the app under stress was established when it handled sudden rush without breaking down. It was the point when the limits were discovered, and this is a critical point when knowing how to make it work safely and correctly. The security checks were not set in the initial stages; they were added into the process subsequently. The interaction between the users is done through a site in which they control what they write. The ability to test unusual formats was a way to test its tolerance to errors, such as lengthy entries, undesired strings, and faulty API calls. Wrong entries were blocked in order to conceal mistakes and file structures. Resaving the files

ensured that the warning information was not deleted in the app file. Because the permission settings were checked as the users had been accessed to the stored records. When I used the software with admin privileges it was able to handle missing access properly. In case anything goes wrong, there would be clear warnings that would show me the potential fixes without revealing anything personal. The technical checks, which were done in form of testing, gave useful insights on actual users. The ability to switch sections quickly was examined with the importance of the clarity of alerts. Critical warnings, filter setting, and data visual study were tested by the users as well. Their response identified essential aspects where they could conduct a small change and create a huge impact in the system. Some minor modification of the layout enhanced usability. The most noticeable thing was the smooth movement in actual habit. Repeated instances of old tests occurred during the building exercise. In case something was altered or added, previous checks were made to ensure nothing was broken. Change by doing the same process served to create trust gradually. Machines replaced some of the functions of checking what was happening in the back-end where it was logical to do so. Automated runs were supported by human eyes that were going to be used to examine what machines could not. New checks added with every update and prevented more risks/issues. With the software running, days were crossed to determine the stability. At that period memory usage and task management was monitored. Minor glitch were spotted and were corrected until something went awry. Eventually, all that remained was the files and the performance became sluggish. All the steps were noted down and small corrections made where necessary. All the phases had a certain goal and presented outcomes. The steps may be re-done in case of need. Testing was done on the actual usability as far as safety is concerned, thereby nothing was left unnoticed. Much consideration was attached to the manner of doing things. The strategy kept all things in place and ensured new new updates were less disintegrable and easy to maintain.

6.3 Test Case Design

To determine the functionality of Good AID-SOC Dashboard we designed the tests that would verify each of the main parts step-by-step. The tool tracks the real-time activity of the attempts to make logs and sends warnings that require actions; this means that each of the functions should be reviewed carefully. We considered accuracy, consistency, and speed under load rather than only one of the goals in each test. Any scenario also possessed its ID, purpose, setup requirements, actions to perform, anticipated outcomes, and actual

discoveries. This approach permitted to execute the tests, which served to confirm that nothing went wrong due to some changes. Since the beginning, our tests were centered on the Log Ingestion Module, as it fulfills all the other operation monitoring functions. In this, it is vital that threats will be detected in case the logins are not captured. These tests were used to prove when the successes of logins occurred and when failures and locked accounts occurred. A system would not go back to old logs at a reboot; this allows everything to be organized and flexible. Detection of duplicates was one of the key steps that were deemed important as much as collecting new data. The input system was found to be stable even during frequent test runs as hundreds of events were fed into the system. In cases where there were no or the service was not online, the Windows Event logs would be checked to know how the system would react. Beginning with false alarms, tested the effectiveness of the system in the detection of problems. As opposed to actual attacks, a mock attack like repeating a log in, entering numerous passwords at a time, abnormal sign-in times, or excessive successive checks was made. It is not until a certain limit is reached that the alerts are triggered. The frequent activity of users remained unnoticed, and it never alerted. This balance played a key role in minimizing unwarranted alarms and keeping back the confidence of the detection process. In the middle of testing, AI Scorer Module undergoes tests of how it predicted abnormal patterns and some occurrences. Past events are required before any scoring is done, thus the first trials were not done until sufficient past events had been generated. Unusual behaviour was added to test whether the module rated them in the same way we considered it to be. Next paid attention to the way the levels were computed, and they should be stable even in the case of little examples. Timing was also a factor that was observed and noteworthy as far as, each analysis was performed within an appropriate time slot to ensure that it does not affect the live tracking. Alert Manager related to ensuring that alerts were created, stored, recalled and edited when there was need. After every simulated attack being repeated several times, the system maintained the presence of its JSON data safely by avoiding redundant warnings. The succeeding phases such as New, Acknowledged, or Resolved progressed without any hitches in the lifecycle checks. In the event that there were too many actions trying to read and write simultaneously, precautionary measures turned on to make sure that no conflict arose. It was also checked on the functionality of the web dashboard and its user-friendliness. They also did not only measure speed, but also the speed of the response of APIs, as well as live updates and charts. The accuracy in terms of changing pages was also checked. The look was similar in browsers due to the checks of the layout that modify that. The design was good enough and

not problematic in both large and small devices. The system did not slow down as it was supposed even when there were hundreds of alerts saved. During observation of memory, 24 hours went by without anything concerning problems. Subsequently, the activity demonstrated the use of CPU when there was high demand. On long runs, system resources were given prominence and showed their performance. The dashboards were also smooth even as it continued working hours without any indication of change. No single check was done only once but rather it was repeated so that there was consistency and accuracy. Jobs shifted between beginning and ending. Several simultaneous logins were performed; the response times were not slowed down. The use of the resources was not exceeded even during multiple days testing.

Wrongly structured queries plus harmful code bits got sent on purpose - just to see if the app would block them without crashing. Checking folder and file rules showed whether saved alerts stayed locked away from unwanted eyes. Each round of probing made it clearer: the system holds up well when pushed hard by bad actors.

A snapshot of sample tests from every module appears below. Each entry reflects typical scenarios examined during evaluation

Test Case ID	Module	Objective	Expected Outcome	Result	Priority
TC-01	Log Ingestion	Collect successful login event	Event captured and queued for processing	Pass	Critical
TC-02	Log Ingestion	Prevent duplicate event processing	Event processed only once	Pass	High
TC-03	Log Ingestion	Handle high event volume	All events processed within 5 sec latencies	Pass	High
TC-04	Detection Engine	Detect brute force pattern	Critical alert generated	Pass	Critical
TC-05	Detection Engine	Avoid false positives	No alert for normal login	Pass	Critical
TC-06	AI Scorer	Detect anomaly	Anomaly score	Pass	Critical

		using Isolation Forest	above threshold		
TC-07	AI Scorer	Calculate confidence correctly	Score between 85–95% for strong attack	Pass	High
TC-08	Alert Manager	Store alert in JSON	Alert stored with valid structure	Pass	Critical
TC-09	Alert Manager	Prevent duplicate alerts	Single alert created	Pass	High
TC-10	Web Dashboard	Display real-time alerts	Alert visible without manual refresh	Pass	Critical
TC-11	Web Dashboard	Validate API responses	HTTP 200 with valid JSON	Pass	Critical
TC-12	Performance	Measure processing latency	< 3 sec averages	Pass	Critical
TC-13	Security	Validate input sanitization	Malicious inputs rejected	Pass	Critical

Table 6.1: Major Test Cases

Beginning with clear layouts, every test case covered how functions behave when active. Real-time responses were checked through timed inputs instead of assumptions. Scoring by artificial intelligence went under scrutiny using varied examples across sessions. Stored information stayed intact after repeated access cycles, verified step by step. Navigation on the dashboard unfolded smoothly even under heavy loads. Security checks ran alongside feature trials without delay. Modules first faced isolated evaluations before joining others in combined runs. Behavior remained steady once systems linked together. Results from broad testing show stability where it counts - during actual use. Performance held firm; protections stayed strong throughout each trial phase.

6.4 System Implementation Strategy

Building the AI-SOC Dashboard followed a step-by-step path, one that kept things steady while allowing room to adapt along the way. Instead of launching straight into full

complexity, progress moved piece by piece - each stage checked and confirmed before adding more.

- **How We Build and Apply Ideas**

This stage includes the construction of the Base and the Basic Systems. We began preparations of the coding space and the framework. A different Python workspace could be used so that tools would be similar everywhere. What worked first? Extracting information out of Windows Security Event Logs. You had to have the admin privilege and close attention to the Windows API in order to obtain the login records. Past tests were aimed at the successful drawing of events; that was only the next filter added to the results, making them reduce the number of sign-in attempts, refused access, and frozen accounts. At this time, the structural input section had log input, which was plain basic. The events came down and presented to us on the screen. Upon the runaway, upgrades started extracting information as to who logged in at the time it happened, what occurred, and how it happened. Each information was reliable, and then succeeded by another one in sequence. Data saving comes in the form of JSON files which mean avoiding heavy databases. Due to the fact, it required fewer efforts, and one could see the problems more noticeably when reading pure text logs.

- **Building the Base and Essential Systems**

We started getting the coding space ready along with the framework. A separate Python workspace so tools would work the same everywhere.

What worked first? Pulling data from Windows Security Event Logs. To get the login records, you needed admin rights and careful work with the Windows API. Earlier tests focused on successfully pulling events; only after that did filters appear to narrow results like sign-in attempts, rejected access, and locked accounts.

Currently, the log input part remained basic by design. Events came in and showed up on screen for us to check. After things ran, upgrades began pulling out details, like who logged in when it happened, what event occurred, along with login method. Step by step, each information proved reliable before anything new followed.

JSON files for saving data meant skipping heavy databases. Because of that, things took less effort, issues showed up more clearly when looking at raw text logs. When the system

shuts down without warning, complete file damage is less likely because of the overwrite methods used during saves.

- **Second Stage Detection Systems and Processing Core**

Once events were flowing steadily, attention turned toward spotting patterns. Not long after, rules took center stage in analysis. These came first because they offered clear outcomes, steady and known. Only later did systems begin folding in AI methods that weigh likelihoods rather than certainties.

Midway through development, signs of too many wrong passwords began showing up - each try logged instantly in temporary storage. Alarms were provided immediately those numbers hit a predetermined line. Fine tuning was done later, and not by rushing, but through the use of actual test cases to form levels of response. Not all spikes indicated a case of danger, certain patterns required breathing space before a flag was drawn. The method to identify password spraying is by looking at failed logins across numerous accounts. Not only the attention is paid to the actions of individual users, but also observing their development in all people. Frequent occurrence of the same error in several locations within few minutes is usually indicative of coordination. A combination of these errors would serve to determine when something bigger is occurring. When an account starts failing too many times, this will lock out a user and will be recorded in windows logs. As soon as one of these entries occurs, it links to previous attempts when one has logged in and this happened approximately the same period of time. And this is what correlating desires. This connection includes information about the last time before the logout. Individual mistakes no longer create a clearer picture than each alert makes. Midnight logins and those that are too close together started raising concerns on a new guideline. Incidentally, failure is not triggered by repetition, how fast the attempts are made is of greater importance. Behavioral change has begun to be counted, rather than awaiting any indications. Alarms are also activated not only on the case of overload but also when the action appears to be inadequately aligned. Bringing to the fore the attempts which used not to be noticed.

- **Machine Learning Added**

DBSCAN entered the picture, forming clusters from login behaviors. Without scaling features, the groupings would have been meaningless. With each trial, settings changed and

the results improved when choices prioritized balance.

From rule checks and algorithms came with confidence numbers. Fixed rules clearly identified issues, and their influence was stronger in the displayed results. Anomalies detected by learning systems helped determine the seriousness of each alert. These values were converted into percentages, fitting on displays people check every day.

- **Web Dashboard Development Phase Four**

Starting out, Flask shaped how users interacted with the system through basic page delivery. At first, only fixed screens appeared, which were enough to test navigation flow and visual setup. Over time, live connections developed behind the scenes, pulling in alerts and number summaries as they changed.

Built with repeatable pieces, templates ensured uniformity across pages. Right up front, the dashboard displayed an overview with important numbers and fresh warnings. For quicker reading, colors indicated the seriousness of each alert.

Every few seconds, the front end checked the back end for new data. While not as fast as live streaming, this method kept everything updated. It worked well without requiring extra layers of system design.

Pages like analytics popped up to show what's really happening behind the scenes. Instead of just numbers, graphs revealed how often threats got caught over time. Visual tools made it easier to spot when alerts spiked during certain hours. Seeing data shift across timelines helped understand system behavior more clearly. Charts responded to clicks, letting details unfold without extra steps. Performance clues emerged simply by hovering over colored bars and lines. What once took spreadsheets now appeared in moving shapes on screen. Patterns hid at first glance but showed up once you started poking around. Each view adjusted slightly based on where someone looked first. No two visits felt exactly alike thanks to live-updating visuals.

- **Integration and System Testing Phase Five**

A fresh layer tied silent background checks to the online display. Running nonstop, a separate tracker pulled in updates while leaving site functions free. Shared information stayed intact through careful coordination steps. Each piece moved in step, avoiding clashes where systems met.

Logs now capture every move the system makes, helping spot issues fast. Each message gets a label based on how serious it is. Serious glitches stand out from routine notes. Now users can adjust detection levels along with how long things run, thanks to a built-in option.

Phase Six Performance Optimization

Earlier checks pointed out small fixes. Since specific jobs happened nonstop, bundling helped cut weight. Heavy demand on info meant storing it nearby reduced effort each round. Out of nowhere, performance jumped even though the main design stayed put.

- **Documentation and Deployment Prep**

Because APIs weren't clear, adjustments happened step by step. Testing different login sequences helped shape how the AI responded. Instead of rushing, each pattern went through repeated checks. When multiple processes ran at once, timing became critical. Without proper locks, data could mix up unpredictably. Errors showed up only under specific loads. Synchronization wasn't added early - it had to be retrofitted later. Small delays exposed hidden conflicts. Even slight overlaps caused unexpected behavior. Patterns emerged only after many failed attempts. Stability came slowly, not all at once. Each fix revealed another issue underneath. Progress depended on observing subtle details. What worked alone failed in combination. Consistency grew from repetition, not design. Adjustments continued until mismatches stopped appearing. Eventually, things lined up without constant oversight. No single change fixed everything. Pieces fell into place over time.

Starting small made it easier to handle problems as they came up. Because changes were added step by step, each piece could be tested well before moving on. That way, the system stayed reliable even as more complex parts appeared later. From start to finish, the plan turned an idea into working software that watches for security issues using artificial intelligence.

CHAPTER – 7

CONCLUSION

A half-finished version of the AID-SOC Dashboard now exists, built to track Windows logins. Instead of depending on costly tools, it demonstrates how live, smart notifications, and organized visuals come together through everyday technology. The fascinating thing is that the ideas that are traditionally contained in textbooks actually play out when data of real systems has been used. It reads security logs in windows and deals with making meaning out of sign-in, thereby paying special attention to the sign-in. It pulls logins directly not by use of the operating system but through outdated recordings or forged entries. It is due to this that you now see. feels authentic. All the warnings are based on the something that occurred on the device. Now the system operates like a professional application. Due to this arrangement, the work keeps in touch with the reality rather than theory. What is displayed here makes it very clear how a small-scale system might actually work. AID-SOC is an amalgamation of all that, it makes use of layers - the one that produces the visuals, the one that produces the computation or the one that produces both. storage in itself - but connected when need be. Activities such as logging and threat identification and risk allocation. use of AI to score, alert control, and showing the results on dashboards works flawlessly. Because each part has its role, it is easier to understand and its upkeep is also easier. In case changes occur afterwards such as addition of new. they can be integrated easily without any distraction to other parts as they are detection rules, or cover more types of events. The entire system is expanded on command. Today, what is good is still good next day. The observation of events taking place is a major project success. The tool works behind the scenes and this is the reason why it is so. It is highly significant in this project since catches new login attempts as they come by. As it recalls the most recent. entries it dealt with, aged records are automatically skipped or deleted. In this manner, the process will be efficient. Not to repeat unnecessary steps again and again in order to do it. Even when increased activity comes on board, performance continues to be stable. the system. Numbers in the tests were displayed by a login action almost the next moment and then a matter of seconds later. they were exhibited it is a good thing. System safety is concerned with prompt detection because it will coincide with delays in the identification. problems may result in greater complications in the future. In some cases, their atypical behavior is defined when the user attempts to log in. Normative rules determine familiar flags, including excessive incorrect password attempts, intrusion into

logins of other sites, and account freezes. These checks are still effective in the day-to-day threats. Meanwhile, mathematical models which operate silently keep track of the user behavior, in advance. A combination of clear alerts and numericals will not lose the sharpness. trust. Some warnings are more conspicuous than others, some confidence scores assist in showing which ones would be really significant. This is a step by step filtering that demystifies decisions and eliminates random alerts. Even an effective screen layout enhances the functionality of a system. It sorts instead of showing untidy logs simultaneously. warnings into arranged parts and presentations. Colors denote seriousness, aggregation of figures as well as time. charts help identify risks. Knowledge is current, and requires no human intervention to refresh the information. Clarity and as both are indispensable to successful monitoring. Instead of its dependence on flash graphics, it prefers neat. and easy layouts, so the navigation is simpler. Both in classes and running operation, such simplicity guarantees everything works. What is not decorative but rather the running of the process. On the open technology working demonstrates what there can be developed by a small team with no great budgets. Python works in the background to operate with data and the Windows applications retrieve all the alerts through the operating system. The lightweight web interface is achievable with flask which binds all these together into a format that can be used. Sometimes the in-house tools are useful and everyone does not need paid-source software. Security will not build upon the costly equipment but rather upon the ability to think and the ability to make sure the setup is adequate. Even simple parts work quite well when they are crafted to fit together. On the academic front, the piece links the ideas on cybersecurity with their application to reality. It does not talk about the possibility of tracking the introduction of the device into the system, discovering suspicious activity, connecting the incidents, or controlling notifications, but demonstrates these processes in action. Step-by-step analysis of unfiltered log data will allow learners to have a more profound insight into the system itself, and the process of SIEM recognition. Working involves a lot more than reading passively. The thought within the mind becomes deeper when the theory is put to practice in real life situations. With as much as is possible, there are limits. At this point, it works with Windows alone due to direct retrieval of information in Windows Security Event Logs. New components such as syslog or some other logging configurations would have to be created to run on Linux or even on the macOS. Neither do all of the types of events get attended to, either- currently, only attempts at logging in count. This foreshadowing is too narrow to clutter the work. Nevertheless, file access monitoring would improve the level

of security. Any modification of user privileges might also be a sign of risks. Another layer is formed by network connections. It is equally important when the internal systems settings are changed. Every space that may seem as a weakness is actually an entry-point. The approaches can be addressed separately in future versions.

CHAPTER – 8

LEARNING DURING PROJECT WORK

Practical Understanding of Security Monitoring

It worked with live Windows logs and this experience in working with Windows logs allowed me to realize how real security monitoring works rather than just the theoretical information.

- The manipulation of Event IDs such as 4624, 4625 and 4740 facilitated the authentication events and made them meaningful and practical.
- The study of the sequences of logs gave me the idea of how abnormal behavior conceals within the normal activity. I also had an eye opener in that it is not enough to get logs but it is the actual task to find patterns in logs.
- The timing of analysis in the live log demonstrated how timing has a direct effect on security response effectiveness.

Designing Detection Logic and SOC Thinking

Making detection rules assisted me to reason like a security analyst and not solely as a programmer.

- I also learnt that the failed logins that come up frequently should be done through analysis within time constraints in order to validate the attack patterns.
- Lessening excess alerts was an eye opener on how real SOC teams can curb alert fatigue. I learned the importance of responding both too swiftly and slowly as threats to security.
- Correlation logic construction enhanced my aptitude to interpret related events and not single events.

Web Dashboard and Technical Development

Analysis: I have the Flask dashboard, and it has enhanced my knowledge in regards to backend and frontend communication.

- Work with API endpoints made me understand how the process of communication between the browser and the server works with the help of the JSON data.
- The use of live updates enhanced my knowledge about the REST-based systems.
- The interface design taught me the role of visual simplicity in making a decision.
- The application of the color-coded alert system contributed to the immediate distinction

of levels of severity.

System Performance and Resource Management

- Running background threads helped me understand how multiple systems can be able to run numerous tasks at the same time.
- I was informed that thread management may crash a complete app when it is managed improperly. To observe the effect of optimization of resources on long-term stability, it was possible to monitor CPU and memory usage. - The constant log observed that performance tuning is vital to real time systems.
- The experience of processing delays allowed me to see the trade-off between the speed and the stability.

Understanding Authentication Attacks

Watching actual failed pattern login allowed brute-force attacks to be identified more easily.

- The experience of password spraying also helped comprehend how attackers probe several accounts in the background.
- Lockout incidences served to make me realize how failure to act will become a serious threat with time.
- Abnormal times of logging in had been assigned significant meaning, rather than being an abstract concept. Real log data was presented illustrating the combination of small pieces of information to create larger dangers.

Critical Thinking and Troubleshooting.

- Duplicate processing of events to fix enhanced my logic level of debugging.
- Raising permission errors enhanced my system-level access control knowledge.
- Question: Timestamp mismatches allowed me to sharpen my eye/ear.
- The dashboard synchronization problems made me a better problem solver.
- Complex errors were solved easily by breaking them down into smaller bits.

Modular Design and Code Organization

- The modules have been divided into log ingestion, detection, alert management, and dashboard, which have enhanced maintainability.
- Modular coding ensured that the process of debugging was quicker and risk-free.

- Structured file systems existed and there were no problems with confusion as updates were made.
- Separation of responsibilities was done clearly minimizing system complexity. When designed with structure, growth could be easily accommodated without the need to re-write

Documentation and Project Planning

Writing in depth documentation increased my skill to clarify technical concepts.

- Creation of system diagrams helped understand the relationships between components.
- Division of chapters made me have a deep understanding of the project.
- Planning was done more efficiently with visual representations.
- The project was reproducible and well organized due to proper documentation.

Time Management and Project Discipline

An implementation process that was done in a sequential order enhanced productivity. Continuing to complete modules in quality order helped to minimize confusion.

- Handling of deadlines enhanced my planning.
- The consistency was more important in the development than motivation.
- Systematized workflow enhanced efficiency of the projects.

Role of AI in Transforming the Project

Advantages of AI Integration

Isolation Forest: Isolation forest allowed financial fraud to be detected without the user specifying all the threat rules. DBSCAN was used to automatically discover clustered patterns of attacks.

- Analysis of the manual logs was minimised by AI. People used to prioritize serious alerts over minor ones with the help of confidence scoring. Adaptive learning- The system had been able to improve as it received more data.
- Constant surveillance proved to be non-fatigable to the human.
- AI picked up harmonic behavioral changes invisible to rule-based logic.
- Live scoring anomalies enhanced advance threat detection.
- Automation minimized dull work on the part of analysts.
- AI allowed scaling analysis of the events that could not be analyzed by people.

Limitations and Challenges of AI

- Tuning was done by trial and error. False positives were found in cases where the values of contamination were misconfigured.
- Only, the accumulation of the baseline data resulted in the improvement of the model accuracy.
- There was an increment in resource consumption when there were peak event loads.
- Added metadata was not enough to limit interpretability.
- AI had to be balanced carefully in between sensitivity and precision. It can be associated with the necessity of recalibration through deployment in new environments.

How AI Reduced Human Work

- AI pattern recognition in recent thousands of logs.
- Correlation of events was done automatically and without manual monitoring. Continuous monitoring eliminated the situation where human supervision was necessary.
- Prioritization of alerts shortened the investigation time.
- Monotonous reviewing activities were done away with.
- Massive data processing was made possible.
- The adaptive learning minimized hand rule corrections.
- Identification was performed with lowered workload on analysts.
- Improvement of self-improvement minimized maintenance.
- AI served as an automated security analyst on a first-level basis.

Overall Personal Growth

Based on the project, both technical and analytical thinking were solidified. Practical implementation related theory with practice. I was also able to have confidence in dealing with live security environments.

- The incorporation of AI added to my knowledge of the current cybersecurity tools.
- The experience helped me to be better at problem-solving in case of pressure. Practical work helped me to create my system design mentality.
- Debugging enhanced patience and rational thinking.
- The performance limitations were sensitized to me by real time testing.
- Automation altered my approach towards security issues.
- The project established a good base to work on cybersecurity in the future.

CHAPTER – 9

FUTURE ENHANCEMENTS

1. Future extend the system for Linux and MacOS
2. Implement deep learning techniques
3. Combine multiple AI algorithms for hybrid detection
4. Add correlation engine for multiple stage of attacks
5. Develop a distributed architecture.
6. Can implement cloud-based storage
7. Add internal threat intelligence
8. Expand monitoring for the network
9. Add automated detection and ticketing
10. Replace JSON with other databases

CHAPTER – 10

BIBLIOGRAPHY

10.1 Online References

- [1] I. H. Sarker, A. S. M. Kayes, S. Badsha, H. Alqahtani, P. Watters, and A. Ng, “Cybersecurity data science: An overview from machine learning perspective,” *Journal of Big Data*, vol. 7, no. 1, pp. 1–29, 2020. [Online]. Available: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-020-00318-5>
- [2] F. T. Liu, K. M. Ting, and Z. H. Zhou, “Isolation Forest,” in *Proc. 8th IEEE Int. Conf. on Data Mining (ICDM)*, 2008, pp. 413–422. [Online]. Available: <https://ieeexplore.ieee.org/document/4781136>
- [3] National Institute of Standards and Technology (NIST), “Computer Security Incident Handling Guide,” NIST Special Publication 800-61 Rev. 2, 2012. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>
- [4] Microsoft Security Intelligence, “Password Spray Attacks: Detection and Prevention,” 2023.[Online].<https://www.microsoft.com/security/blog/2020/04/23/protecting-organization-password-spray-attacks/>
- [5] MITRE Corporation, “MITRE ATT&CK®: Enterprise Matrix for Enterprise,” 2023. [Online]. Available: <https://attack.mitre.org/matrices/enterprise/>
- [6] Gartner, “Magic Quadrant for Security Information and Event Management,” 2023. [Online]. Available: <https://www.gartner.com/en/documents/4018415>
- [7] SANS Institute, “SIEM Implementation and Tuning Best Practices,” 2022. [Online]. <https://www.sans.org/reading-room/whitepapers/logging/siem-implementation-tuning-best-practices-39530>
- [8] Elastic N.V., “Machine Learning in Elastic Security,” 2023. [Online]. Available: <https://www.elastic.co/guide/en/security/current/machine-learning.html>
- [9] Pallets Projects, “Flask Web Development Framework Documentation,” 2023. [Online]. Available: <https://flask.palletsprojects.com/en/2.3.x/>
- [10] scikit-learn Developers, “Machine Learning in Python Documentation,” 2023. [Online]. Available: <https://scikit-learn.org/stable/documentation.html>
- [11] Microsoft Corporation, “Windows Event Log API Documentation,” 2023. [Online]. Available: <https://docs.microsoft.com/en-us/windows/win32/eventlog/event-logging>
- [12] Python Software Foundation, “Python Programming Language Documentation,”

2023. [Online]. Available: <https://docs.python.org/3/>
- [13] OWASP Foundation, “OWASP Top 10: Web Application Security Risks,” 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [14] Cybersecurity and Infrastructure Security Agency (CISA), “Cybersecurity Best Practices,” 2023. [Online]. Available: <https://www.cisa.gov/cybersecurity-best-practices>
- [15] IEEE Xplore Digital Library, “Machine Learning for Cybersecurity Research Papers,” 2023. [Online]. Available: <https://ieeexplore.ieee.org>
- [16] GitHub, Inc., “Open Source Security Tools and Projects,” 2023. [Online]. Available: <https://github.com/topics/cybersecurity>
- [17] Amazon Web Services, “AWS Security Best Practices,” 2023. [Online]. Available: <https://aws.amazon.com/security/security-resources/>
- [18] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
[Online]. Available: <https://dl.acm.org/doi/10.1145/1541880.1541882>
- [19] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.
[Online]. Available: <https://ieeexplore.ieee.org/document/7307098>
- [20] M. Ahmed, A. N. Mahmood, and J. Hu, “A survey of network anomaly detection techniques,” *Journal of Network and Computer Applications*, vol. 60, pp. 19–31, 2016.
[Online]. Available: <https://www.sciencedirect.com/science/article/pii/S108480451500289X>
- [21] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, “Survey of intrusion detection systems: techniques, datasets and challenges,” *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.
[Online]. Available: <https://cybersecurity.springeropen.com/articles/10.1186/s42400-019-0038-7>
- [22] H. Liu and B. Lang, “Machine learning and deep learning methods for intrusion detection systems: A survey,” *Applied Sciences*, vol. 9, no. 20, p. 4396, 2019.
[Online]. Available: <https://www.mdpi.com/2076-3417/9/20/4396>
- [23] M. A. Ferrag et al., “Deep learning for cyber security intrusion detection,” *Journal of Information Security and Applications*, vol. 50, 2020.
[Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214212619303002>

- [24] A. Thakkar and R. Lohiya, "Attack classification using feature selection techniques," *Journal of Ambient Intelligence and Humanized Computing*, vol. 12, no. 1, 2020.
[Online]. Available: <https://link.springer.com/article/10.1007/s12652-020-01782-6>
- [25] M. Sarhan et al., "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, 2022.
[Online]. Available: <https://link.springer.com/article/10.1007/s11036-021-01738-9>
- [26] A. Alotaibi and M. A. Rassam, "Enhancing sustainability of deep-learning-based intrusion detection," *Sustainability*, vol. 14, no. 21, 2022.
[Online]. Available: <https://www.mdpi.com/2071-1050/14/21/13794>
- [27] F. Ullah et al., "Explainable artificial intelligence for intrusion detection," *IEEE Access*, vol. 11, 2023.
[Online]. Available: <https://ieeexplore.ieee.org/document/10239014>
- [28] I. H. Sarker et al., "Cybersecurity data science: An overview," *Journal of Big Data*, vol. 7, no. 1, 2020.
[Online]. Available: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-020-00318-5>
- [29] F. T. Liu, K. M. Ting, and Z. H. Zhou, "Isolation Forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
[Online]. Available: <https://ieeexplore.ieee.org/document/4781136>
- [30] M. Ring et al., "Flow-based benchmark data sets for intrusion detection," in *Proc. European Conf. Cyber Warfare and Security*, 2017.
[Online]. Available: <https://ieeexplore.ieee.org> (Search Title)
- [31] G. Apruzzese et al., "On the effectiveness of machine and deep learning for cyber security," in *Proc. IEEE Int. Conf. Cyber Conflict (CyCon)*, 2021.
[Online]. Available: <https://ieeexplore.ieee.org/document/9461092>
- [32] N. Moustafa et al., "Federated TON_IoT Windows datasets," in *Proc. IEEE TrustCom*, 2021.
[Online]. Available: <https://ieeexplore.ieee.org/document/9343047>
- [33] N. Provos and P. Honeyman, "ScanSSH: Scanning the Internet for SSH servers," in *Proc. 12th USENIX Security Symposium*, 2003.
[Online]. Available: <https://www.usenix.org/conference/12th-usenix-security-symposium>

10.2 Offline References

- [34] W. Stallings and L. Brown, *Computer Security: Principles and Practice*, 4th ed. Pearson Education, 2018.
- [35] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [36] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- [37] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.
- [38] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Pearson, 2011.
- [39] R. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed. Wiley, 2020.
- [40] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. Springer, 2017.
- [41] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [42] L. Kaufman and P. J. Rousseeuw, *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley, 2005.
- [43] N. Provos and P. Honeyman, “ScanSSH: Scanning the Internet for SSH Servers,” in *Proc. 12th USENIX Security Symposium*, Berkeley, CA, USA: USENIX Association, 2003.